NPLify · P0 Technical Baseline

Entity-Relationship Diagram & Data Model

New XP Technologies Limited

28 October 2026 · Confidential

How to read this pack

Four map pages are now part of the pack as documents of their own, and are the place to start: the Top-Down Map (from the business, through the data model and the deal states, to the scenarios and the reading order), the Deal Flow on one page (Quote, Collect, Convert, Settle, with the roles and the money path), the Workflow Tree (8 classes, 27 sub-classes, 96 procedures, each naming who initiates, who approves, the codes it is written under and the records it touches) and the Workflow Tree on one page. Read them in that order, then this document. Every worked transaction (T), acceptance vector (L, B), walkthrough (W, X), flow (F) and procedure (C, A, D) has one identifier, listed in the Reading Companion's identifier register; the Test Strategy & Coverage document says which test proves each procedure.

Conventions

Convention: all monetary amounts are stored as exact decimals; every table carries `created_at` / `created_by`; audit, approvals and locking are modelled once (domain 4) and apply to all master data. Derived figures (client balance, group entitlement, offsets, custody) are **views over postings and deals — never stored**.

Seven diagrams across four domains, one schema: 78 tables and 12 views. In each diagram, boxes show the key fields only (full attributes in Appendix B) and are coloured by class — configuration and master data, rate reference, transactional event, ledger and control, derived view; a dashed box is a table drawn in another diagram and names it. Relationship lines run from the referenced row to the row that stores the foreign key; a dashed line is a reference or a polymorphic link resolved by a type column plus an id. The deal lifecycle has three diagrams: 3a is the deal and its collection, 3b the conversion and its exceptions, 3c the settlement. Appendix A reconciles the model with the FRS; Appendix B is the data dictionary.

Scope (NPL decision, 19 October 2026): the whole model is in scope for Phase 1. Nothing among the 78 tables and 12 views of this edition, including the tables added in it (`EMERGENCY_ACTION`, `PARTNER_STATEMENT_RECONCILIATION`, `OPENING_ENTRY`), is deferred; the milestone estimates of the proposal stand.

Classification of the tables (fact and dimension)

Pete asked which tables are the facts and which the dimensions. In this model there are five classes; every table belongs to exactly one, and the diagrams (1a to 4) colour each box by its class. Configuration and master data are the dimensions: rows that describe how the business is set up, versioned by effective date and never edited. Rate reference is a time series of reference values. Transactional events are the facts: one row per thing that happened to a deal. Ledger and control tables are the facts of the books and the records that govern them. Derived views are never stored.

Class	Tables	Count
Configuration & master data (dimensions)	`CLIENT, PARTNER, HOLIDAY, PARTNER_CUTOFF_OVERRIDE, RATE_SOURCE, MARKUP_SHARE_PARTY, SYSTEM_SETTING, MESSAGE_TEMPLATE, PROJECT, THRESHOLD, CURRENCY_PAIR, PAIR_RATE_SOURCE, OWN_WALLET, SENDER, SENDER_RECEIVER_ALLOW, COLLECTION_SENDING_ENTITY, FEE_OVERRIDE, RECEIVER_GROUP, RECEIVER, RECEIVING_ENTITY, RECEIVING_ENTITY_ACCOUNT, WALLET_SCREENING, BANK_ACCOUNT_DETAILS, BANK_DETAILS_TEMPLATE, PARTNER_CONFIG, PARTNER_PAIR, FEE_STRUCTURE, FEE_TIER, PARTNER_ENTITY, COLLECTION_RECEIVING_ENDPOINT, SETTLEMENT_SENDING_ENTITY, SETTLEMENT_RAIL, DESTINATION_APPROVAL, MARKUP_SHARE_RULE, PARTNER_REBATE_RULE`	35

Class	Tables	Count
Rate reference	`MARKET_RATE`, `RATE_COMPARISON`, `PARTNER_RATE_VERSION`	3
Transactional events (facts)	`INVOICE`, `DEAL_GROUP`, `DEAL`, `QUOTE_PACKAGE`, `QUOTE_VALIDITY_EXTENSION`, `CUSTOMER_APPROVAL`, `CUSTOMER_APPROVAL_EVENT`, `SENDER_CREDIT`, `DEAL_FEE_TIER`, `COLLECTION`, `CONVERSION`, `REROUTE`, `EARNINGS_RECEIVABLE`, `BALANCE_CONVERSION`, `ENTITLEMENT_REATTRIBUTION`, `EARNINGS_OFFSET`, `SETTLEMENT`, `SETTLEMENT_LINE`, `SETTLEMENT_RETURN`, `CONFIRMATION`, `SHORTFALL`, `BANK_FEE_EVENT`, `FEE_DECISION`, `MARKUP_SHARE_ACCRUAL`, `PARTNER_REBATE_ACCRUAL`, `LOSS_EVENT`, `LOSS_RECOVERY`	27
Ledger & control (facts of the books; governance)	`LEDGER_TRANSACTION`, `LEDGER_POSTING`, `LEDGER_ACCOUNT`, `OPENING_ENTRY`, `PARTNER_STATEMENT_RECONCILIATION`, `USER`, `ROLE`, `ROLE_RIGHT`, `APPROVAL`, `AUDIT_LOG`, `RECORD_LOCK`, `EMERGENCY_ACTION`, `TYPE_REGISTRY`	13
Derived views (never stored)	`PROJECT_BALANCE`, `GROUP_ENTITLEMENT`, `OFFSET`, `CUSTODY`, `INVOICE_BALANCE`, `PROJECT_MONTH_VOLUME`, `CLIENT_POSITION`, `WALLET_REGISTRY`, `REPORTING_VALUE`, `RESELLER_MARGIN`, `REMITTANCE_MARGIN`, `NPL_PROFIT`	12 views

78 tables and 12 views. `WALLET_SCREENING` is configuration-domain data although Operations records it, because its decision binds later deals on the same address; `PARTNER_RATE_VERSION` is rate reference although Finance confirms it, because deals stamp it as a reference, not as an event of their own.

1 · Configuration domain

A **Client** is the commercial relationship NPL contracts with; a **Project** is one arrangement with it and owns its configuration. Partners are global counterparties; how a partner is used *in this project* is the **partner configuration** block — fees, vehicles, endpoints, destination approvals and rebate terms hang off it. Counterparty receivers are organised in **receiver groups**, the unit entitlement is tracked on, and each receiving entity holds one or more **accounts** with a currency and rail.

Diagram 1a. PARTNER, HOLIDAY, PARTNER_CUTOFF_OVERRIDE are defined in Diagram 1b.

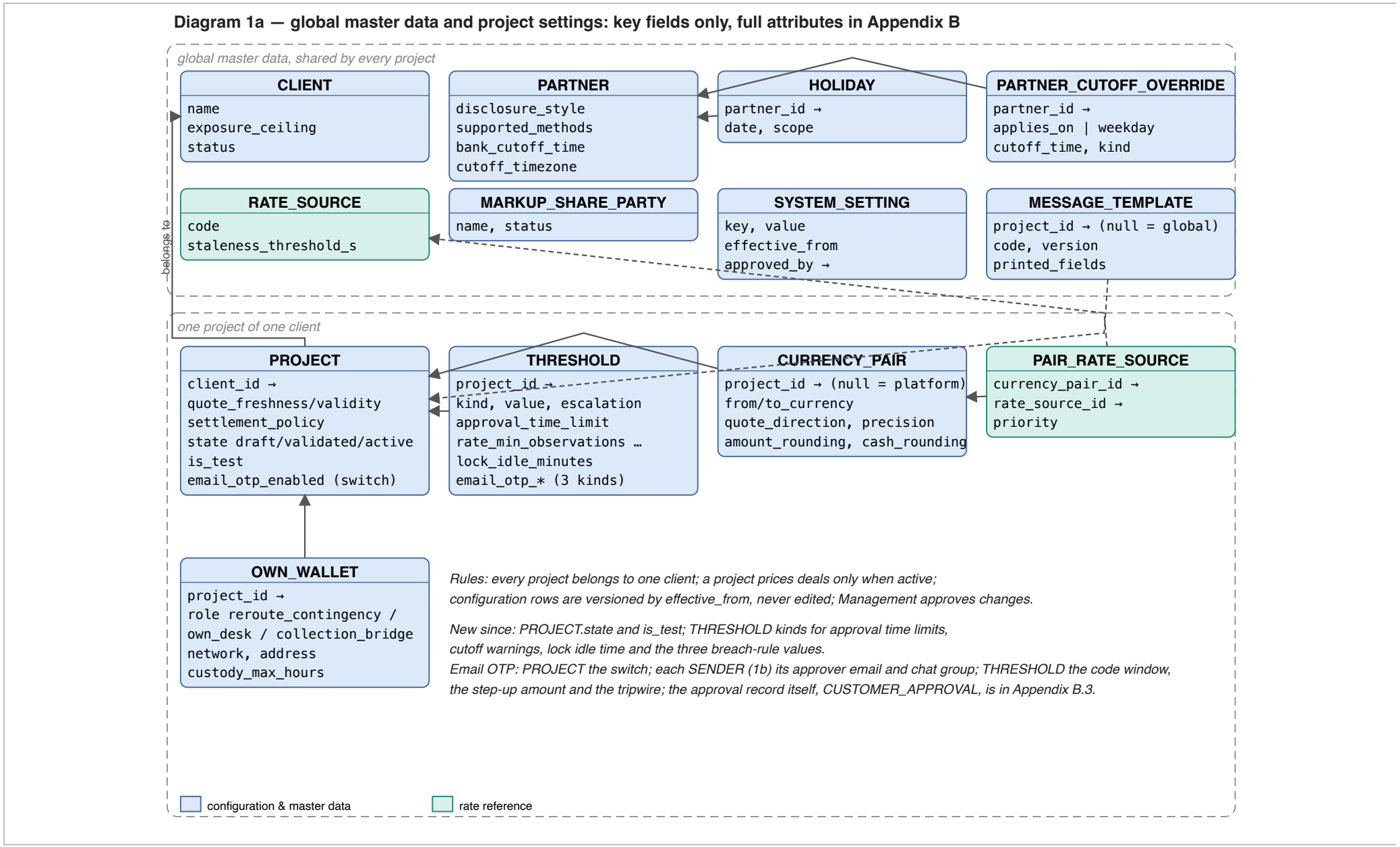


Diagram 1a — Configuration — global master data and project settings

Diagram 1b. `PROJECT`, `CURRENCY_PAIR`, `OWN_WALLET` are defined in Diagram 1a.

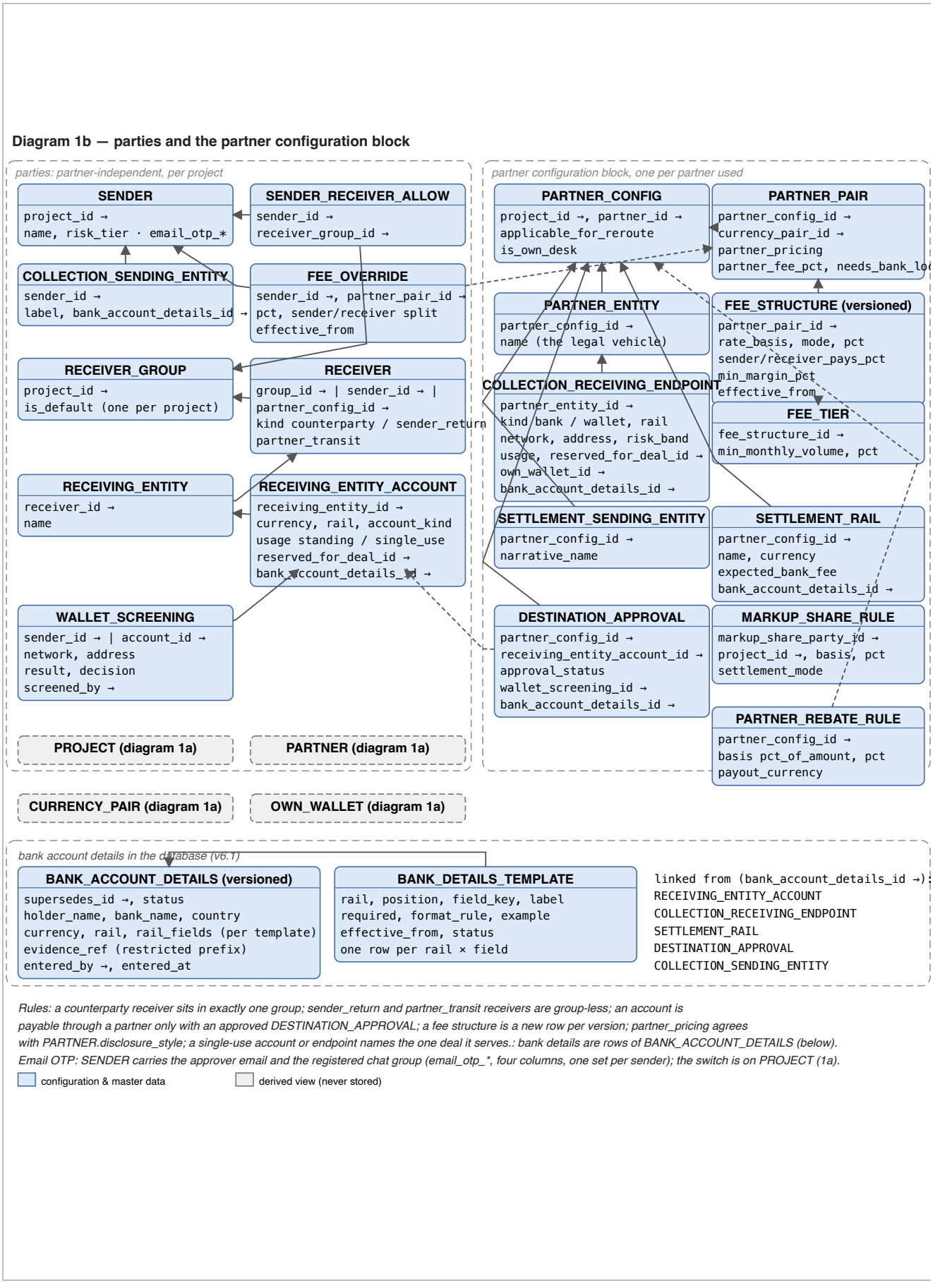


Diagram 1b — Configuration — parties, the partner configuration block, fees and tiers

Notes

- A **Client** is the commercial relationship NPL contracts with; a **Project** is one arrangement with it and owns its configuration. Counterparty receivers sit in **receiver groups**: every project has one default group and every counterparty receiver belongs to exactly one group. Entitlement is tracked per group, so payouts within a group are ordinary settlement and only payouts across groups are offsets; a project with a single group never has an offset.
- A **Receiver** has a kind: `counterparty` for a client receiver; `sender_return` stands for a sender so refunds use the ordinary payout path and the partner's destination approval; `partner_transit` stands for a partner's collection endpoint so the hop of a two-leg route is an ordinary payout. `sender_return` and `partner_transit` receivers belong to no group (`group_id` is null). They never appear on forward deals or allow lists — both validate kind = counterparty — and a payout to them is attributed to the group of the leg that pays them, so no entitlement moves between groups on a refund or a hop. Groups exist for entitlement and offset accounting; refund exclusion is a receiver-kind rule, not a group.
- A receiving entity holds one or more **accounts**, each with a currency and rail (EUR via SWIFT and EUR via SEPA are two accounts). An account is a bank account or a wallet (`account_kind`): a bank account carries `bank_account_details_id` and a bank rail; a wallet carries `network` and `address` explicitly, mirroring the partner-side `COLLECTION_RECEIVING_ENDPOINT`, so addresses can be checksum-validated, checked against the rail, and found when the same address is registered with two partners. Destination approvals and settlement lines point at the account. The bank details themselves are rows of `BANK_ACCOUNT_DETAILS`: a bank account carries `bank_account_details_id`, the current version of its details. A details row is never edited — a change is a new row that names the one it supersedes — so a settlement line released before the change still resolves to the details it was released against. The fields a rail needs (IBAN and BIC for SEPA; account or IBAN, BIC, bank address and intermediary for SWIFT; sort code and account number in the UK; bank code and account number on a local rail) come from `BANK_DETAILS_TEMPLATE`, which also carries the format rule each field is validated against on save. Only the evidence (bank letter, KYC, legal document) stays a file under the restricted prefix.
- Fees are per partner × pair (`FEE_STRUCTURE` under `PARTNER_PAIR`), so choosing the partner is part of pricing. `PARTNER.disclosure_style` records whether the partner's markup is visible; `partner_fee_pct` is known only for fee_on_market partners.
- A partner operates through one or more legal vehicles (`PARTNER_ENTITY`: PT-tour, PT Global Inc); the deal stamps the vehicle at quote so paperwork names the right company. Each vehicle owns the endpoints a sender can pay into (`COLLECTION_RECEIVING_ENDPOINT`): kind = bank with a currency and `bank_account_details_id`, or kind = wallet with `network` and `address`. The partner side therefore mirrors the client side — `PARTNER_CONFIG` → `PARTNER_ENTITY` → `COLLECTION_RECEIVING_ENDPOINT` as `RECEIVER` → `RECEIVING_ENTITY` → `RECEIVING_ENTITY_ACCOUNT`. An endpoint also carries a nullable `rail`: a bank endpoint must have it before it becomes active, and a wallet may leave it null but must match `network` when it is set; the endpoint's kind must match the leg's `collection_method`. Cash-in has no endpoint record; the leg carries the city and the collection the token. The collected cash amount is rounded by the pair's `cash_rounding` option — truncate, nearest or round up — to `cash_rounding_unit` (100 / 500 / 1,000 by currency), the same menu as payout rounding.
- Destination approval is per partner: `DESTINATION_APPROVAL` is an account of the project's receiving entity as registered with this partner, carrying the partner's approval status and the payout details the partner holds. Those details follow the account's kind: `bank_account_details_id` (the details as registered with this partner) for a bank account, `network` and `address` for a wallet. `SETTLEMENT_SENDING_ENTITY` is the narrative on an outgoing transfer; `SETTLEMENT_RAIL` is the bank/EMI that executes it; neither holds money. `COLLECTION_SENDING_ENTITY` is provenance only and never a payout destination; its source account, when known, is a `BANK_ACCOUNT_DETAILS` row too.

- OWN_WALLET has a role: reroute contingency (AQN-LT-Sub at Aquanow), own desk, or a collection bridge on a network a partner lacks, forwarded to the partner as a hop within the wallet's custody limit; PARTNER_CONFIG.applicable_for_reroute defines the alternate-partner set. PARTNER.supported_methods declares what a partner offers — cash and local deposits have no standing endpoint to read it from. WALLET_REGISTRY lists every known address with its owner and role. Under an **own-desk** partner configuration (is_own_desk, the NPL-GR game-reseller project) NPL itself is the converting party: its pairs are priced market plus NPL's markup and its collection endpoints are NPL's own wallets (COLLECTION_RECEIVING_ENDPOINT.own_wallet_id); the custody view reports every such balance.
- **Fees.** A FEE_OVERRIDE may carry its own sender / receiver split instead of inheriting the structure's proportion. FEE_TIER rows under a structure price by the project's calendar-month volume; a leg that takes the month past a threshold is split across two tiers. customer_facing_fee_basis / customer_facing_fee_pct on the structure say what the customer's quote prints when it differs from the recorded parts; the printed "source rate" is derived at quote, never typed. CURRENCY_PAIR.amount_rounding is a menu — truncate, nearest or round up, to the unit or to rounding_unit — and a pair may have the same currency on both sides.
- **Who else earns on a deal.** MARKUP_SHARE_PARTY / MARKUP_SHARE_RULE record a share NPL pays — or, with basis share_of_net_markup and party_kind client_party, the customer who provides a sub-account and shares NPL's markup (Raeen): the base is the leg's fee parts less its partner cost, rebates excluded, several parties may hold rules with different shares, and settlement_mode party_retains means the party holds the converted money and pays NPL the remainder monthly. PARTNER_REBATE_RULE records the rebate a partner pays NPL, with a basis (a percentage of the amount converted) and an optional scope by sender or fee structure; the rebate is NPL's alone and is reported as its own stream.
- **Project state and test twin.** A project is priced against only when it is active (Configuration Schema 1a: draft, validated, active); PROJECT.is_test flags the dummy twin project used for scripted test deals, which the clean-target extract of the cutover leaves behind together with every row that belongs to it: the dependency closure and the checks on the production image are in Appendix C. THRESHOLD rows also carry the time limits per approval stage (approval_time_limit), the cutoff warnings (cutoff_warning_min), the lock idle time (lock_idle_minutes) and the three breach-rule values (rate_min_observations, rate_fallback_spread_abs, rate_abs_tolerance).
- **Cutoff, screening, single use.** PARTNER.bank_cutoff_time (the partner's own clock, beside its rate expiry and holiday calendar), its per-day changes in PARTNER_CUTOFF_OVERRIDE (an agreed extension or an early close) and PARTNER_PAIR.needs_bank_lock drive same-day or next-day conversion and, on bank rails, same-day or next-day settlement. Cash legs carry their city (DEAL.cash_location) and cash rate versions theirs. WALLET_SCREENING records the screening of a sending or receiving wallet address, by Operations, and the endpoint's risk_band says which decisions may land on it. An endpoint or a receiving-entity account may be single_use, reserved for one deal (ad hoc local deposit accounts). HOLIDAY rows under the partner are the dates behind every "next business day"; MESSAGE_TEMPLATE is the versioned wording of outgoing messages and declares the fields it may print — a quote template prints sender-facing figures only.

2 · Rates domain

Diagram 2. RATE_SOURCE, CURRENCY_PAIR are defined in Diagram 1a; PARTNER_PAIR in 1b; DEAL in 3a; CONVERSION, BALANCE_CONVERSION in 3b.

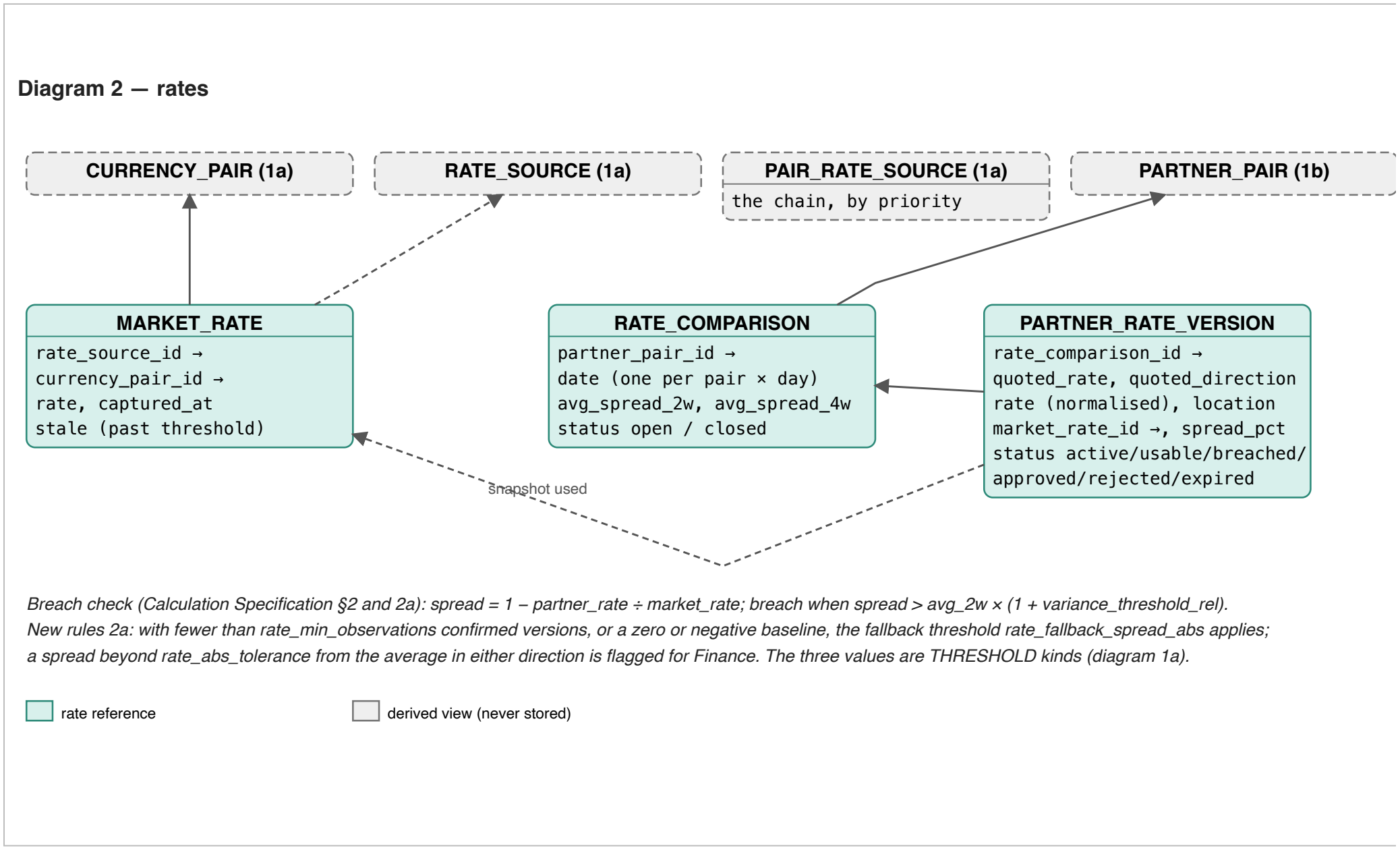


Diagram 2 — Rates

Notes

- The rate comparison for the day is created lazily by the first deal that needs it, one per partner × pair × date, and reused by every deal that day. A partner may give a new rate during the day, so one date can hold several versions. Rolling averages are built from confirmed versions only.
- A version is breached when spread > avg_2w × (1 + variance_threshold_rel); quoting and conversion on that partner × pair freeze until Finance approves with a reason, rejects, or asks for a better rate. Versions are never overwritten; every deal leg, conversion and balance conversion references its exact version.
- Breach rules for thin or unusual history (Calculation Specification 2a): with fewer than `rate_min_observations` confirmed versions, or a zero or negative baseline, the fallback threshold `rate_fallback_spread_abs` applies; a spread beyond `rate_abs_tolerance` from the average in either direction is flagged for Finance. The three values are `THRESHOLD` rows of the project.
- Cross-source slippage is a query over `MARKET_RATE` across sources for the same pair and window.
- A version stores the rate **as the partner quoted it** (`quoted_rate`, `quoted_direction`); `rate` is normalised to the pair's direction and derived — 1 ÷ quoted when the directions differ — so Ali's 1.1738 becomes 0.851934 without a hand inversion, and a direction slip is caught by the breach check. Cash pairs are priced per `location`: INR cash in Delhi and in Mumbai are two versions on the same day.

3 · Deal lifecycle domain

3a · Deal group, legs, pricing stamps, invoice, collection

Diagram 3a. PROJECT, MESSAGE_TEMPLATE are defined in Diagram 1a; SENDER, RECEIVER_GROUP, RECEIVER, PARTNER_PAIR, PARTNER_ENTITY, FEE_STRUCTURE, FEE_OVERRIDE, FEE_TIER, COLLECTION_SENDING_ENTITY, COLLECTION_RECEIVING_ENDPOINT, WALLET_SCREENING in 1b.

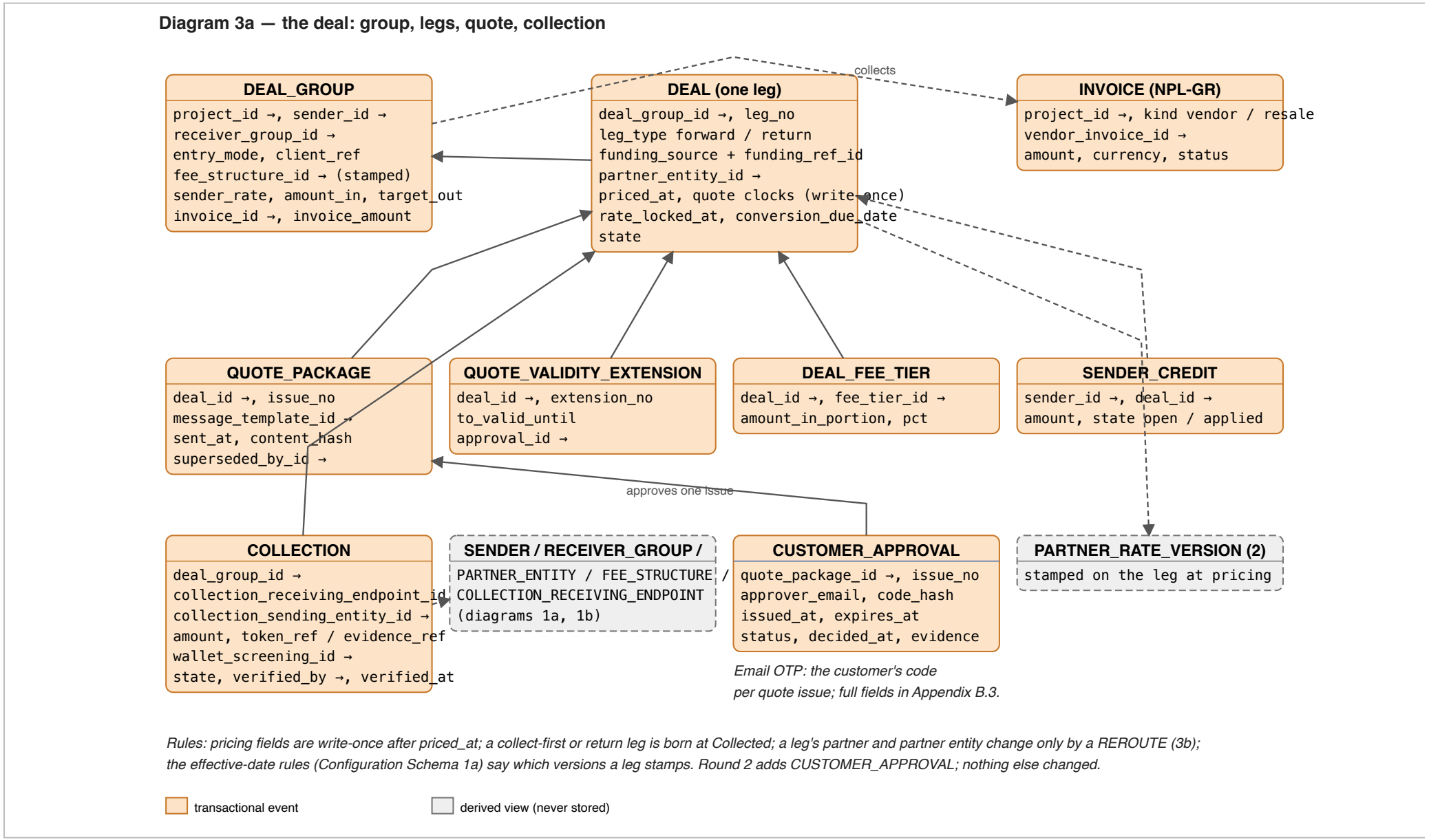


Diagram 3a — Deal lifecycle — deal group, legs, pricing stamps, invoice, collection

3b · Conversion, reroute, earnings receivable, balance conversion, entitlement re-attribution

Diagram 3b. `PROJECT`, `OWN_WALLET` are defined in Diagram 1a; `RECEIVER_GROUP`, `PARTNER_CONFIG`, `PARTNER_PAIR` in 1b; `PARTNER_RATE_VERSION`, `MARKET_RATE` in 2; `DEAL` in 3a.

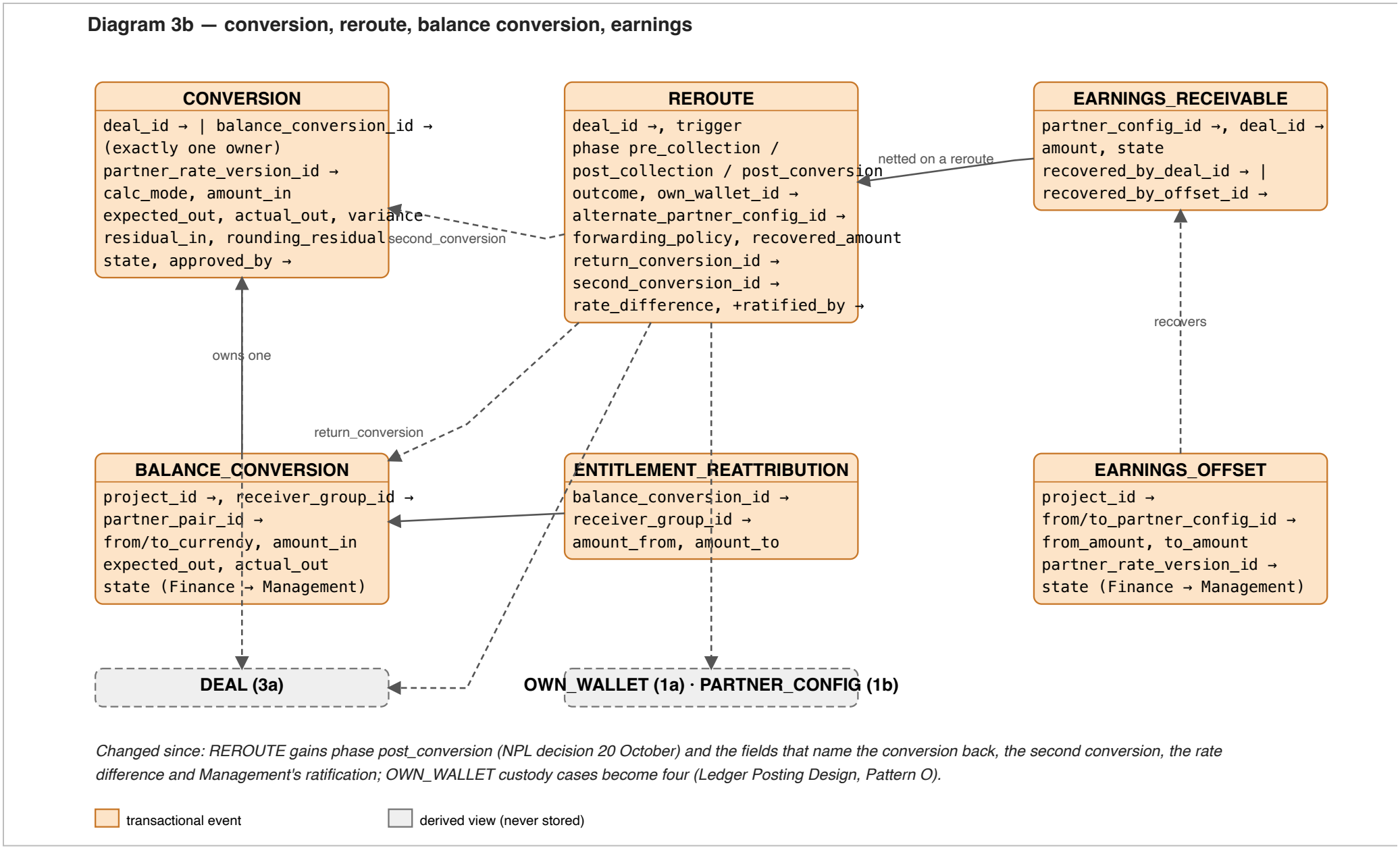


Diagram 3b — Deal lifecycle — conversion, reroute, earnings receivable and offset, balance conversion, entitlement re-attribution

3c · Settlement, return, confirmation, fees, shares, rebates, loss events and recoveries

Diagram 3c. PROJECT are defined in Diagram 1a; PARTNER_CONFIG, SETTLEMENT_SENDING_ENTITY, SETTLEMENT_RAIL, DESTINATION_APPROVAL, RECEIVER, MARKUP_SHARE_RULE, PARTNER_REBATE_RULE in 1b; MARKET_RATE in 2; DEAL, COLLECTION, INVOICE in 3a; BALANCE_CONVERSION in 3b.

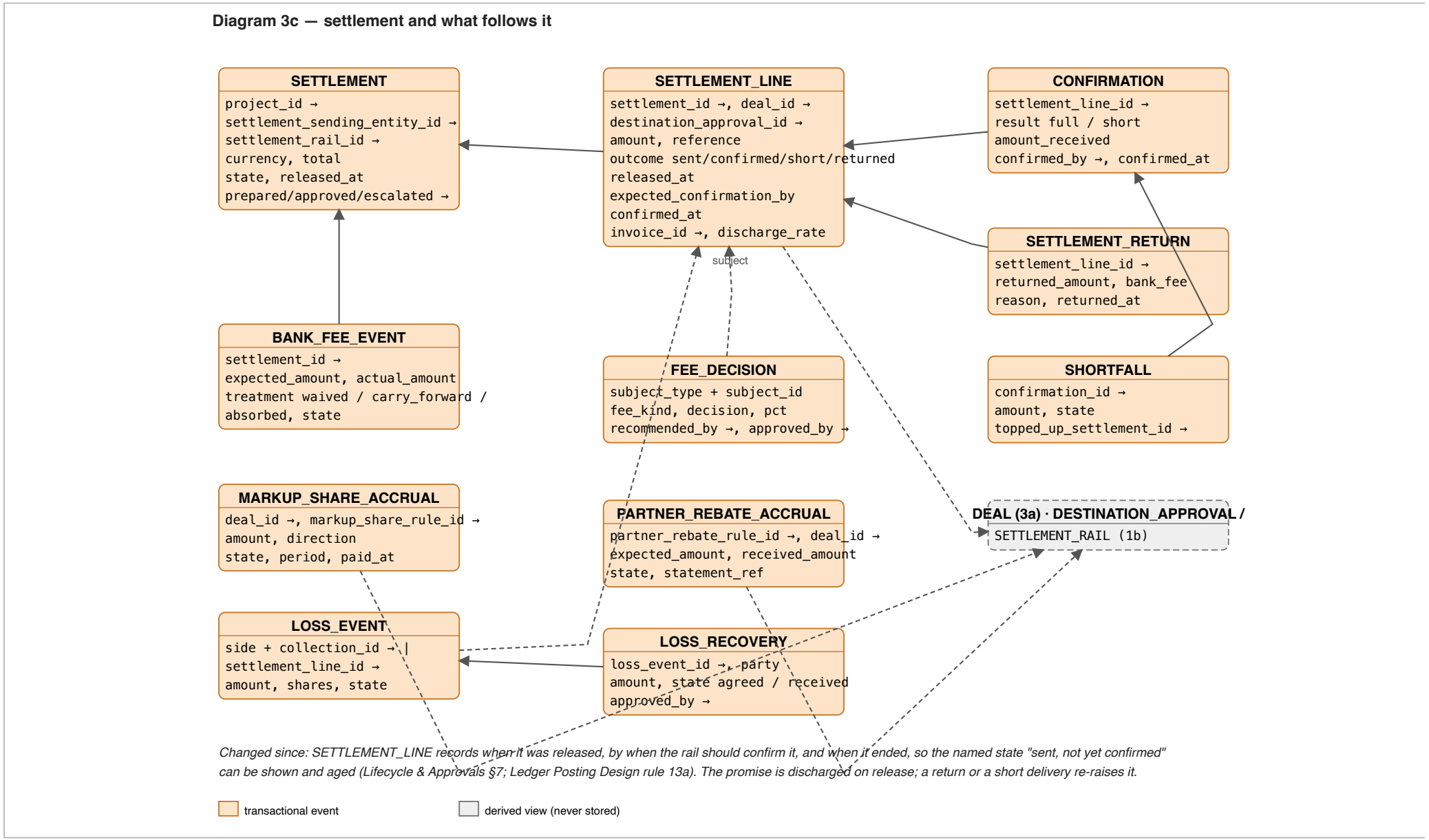


Diagram 3c — Settlement, return, confirmation, fees, shares, rebates, loss events and recoveries

Derived views (never stored)

View	Definition
`PROJECT_BALANCE(project, partner, currency)`	PROJECT_BALANCE(project, partner, currency) = the balance of CL.<PARTNER>.<CCY>.DUE for the partner configuration: Σ debits – Σ credits over postings — conversions in, top-ups, make-goods, offsets and hops in, releases and offsets out — never a sum over deals (the definition summed conversions' actual_out, which includes NPL's margin, and had no term for the events). Was CLIENT_BALANCE. Also reports a pending column — the expected_out of legs whose rate is locked with the partner but whose money has not yet been collected or converted; a void removes it; pending is never a posting.
`GROUP_ENTITLEMENT(group, currency)`	GROUP_ENTITLEMENT(group, currency) = Σ credits – Σ debits over CL.PROJECT.<CCY>.PAYABLE postings carrying receiver_group_id = the group: promises raised at collection and conversion, re-raised on a return or a short confirmation, re-denominated by an offset or a balance conversion, reduced by a release or a client loss share. A payout to a group-less receiver carries the paying leg's group (SETTLEMENT_LINE.deal_id). Σ over a project's groups = the PAYABLE balance = Σ client assets, per currency.
`OFFSET(group, currency)`	OFFSET(group, currency) = GROUP_ENTITLEMENT; positive = owed, negative = holding another group's flow. Arises only when a counterparty receiver is paid from a leg attributed to another group; zero by construction in a single-group project. Carries Management approval and tighter aging.
`CUSTODY(own_wallet)`	CUSTODY(own_wallet) = Σ collections into the own wallet under open reroutes + Σ recovered dues resting there + Σ own-desk client balances + Σ bridge collections not yet forwarded – Σ forwarded or paid out; each row with deal, reason, role and age. The standing control on money in NPL's hands.
`INVOICE_BALANCE(invoice)`	INVOICE_BALANCE(invoice): for a vendor invoice, INVOICE.amount – Σ SETTLEMENT_LINE.obligation_discharged over released lines not returned (a payment discharges on release); for a resale invoice, INVOICE.amount – Σ DEAL_GROUP.invoice_amount over the groups that collected it with a verified collection. Σ VENDOR_PAYABLE = Σ vendor-invoice balances; Σ RESALE_RECEIVABLE = Σ resale-invoice balances (
`PROJECT_MONTH_VOLUME(project, month, currency)`	PROJECT_MONTH_VOLUME(project, month, currency) = Σ DEAL.amount_in over forward legs of the project priced from the opening date, plus the approved opening line of the month in which the project went live, in the tier's volume currency. Read at quote to choose the FEE_TIER; stamped on the leg as tier_volume_basis.
`CLIENT_POSITION(client, currency)`	CLIENT_POSITION(client, currency) = Σ over the client's projects of PROJECT_BALANCE (per partner), GROUP_ENTITLEMENT, CUSTODY, and the period's earnings (EARN_GROSS) and rebates (EARN_REBATE) from postings — the client-level report that project = client made impossible. Compared with CLIENT.exposure_ceiling when set. Its one-currency total is a REPORTING_VALUE, an estimate at the latest market rate.

View	Definition
`WALLET_REGISTRY(network, address)`	WALLET_REGISTRY(network, address) = every wallet address the model knows, with its owner and role: the partners' receiving wallets (COLLECTION_RECEIVING_ENDPOINT kind wallet, with risk_band), NPL's own wallets (OWN_WALLET with role), receivers' wallets (RECEIVING_ENTITY_ACCOUNT kind wallet, with their DESTINATION_APPROVALs) and the sender addresses seen in WALLET_SCREENING with their latest decision. A view, never a second copy of an address: it answers which networks a partner accepts, which wallet is the bridge, and whether an address was seen before under another owner — the FRS 'wallet registry'.
`REPORTING_VALUE(subject, currency, as_of)`	REPORTING_VALUE(subject, currency, as_of) = any per-currency figure of the model — a PROJECT_BALANCE, a CLIENT_POSITION line, a month's collection volume (PROJECT_MONTH_VOLUME), earnings and rebates from postings, custody — multiplied by the latest MARKET_RATE of the platform pair for that currency (its PAIR_RATE_SOURCE chain), with the market_rate_id and captured_at it used and a stale flag when the rate is older than reporting_rate_max_age_min. An estimate for display: the figures in each currency stay the facts, the converted total is a view over them at the moment it is read. Serves project-level and overall collection volumes, earnings and current balances in one currency.
`RESELLER_MARGIN(resale_invoice)`	RESELLER_MARGIN(resale invoice) = resale INVOICE.amount – the amount of INVOICE.vendor_invoice_id, in the invoice currency, equal to RESELL_REVENUE – RESELL_COST posted for the pair: NPL-GR's game reseller revenue, fixed when the two invoices are posted and untouched by anything the remittance side does (;
`REMITTANCE_MARGIN(deal)`	REMITTANCE_MARGIN(deal) = EARN_GROSS (net of markup shares) – EXP_PARTNER ± VAR_FX_TIMING – EXP_BANKFEE – EXP_NETWORK – EXP_ROUNDING on the deal's postings, in the converted currency: NPL's remittance earnings on one deal, including the forex result of instalments paid at different days' rates.
`NPL_PROFIT(scope, period, currency)`	NPL_PROFIT(scope, period, currency) = remittance earnings (EARN_GROSS + EARN_REBATE – EXP_PARTNER – EXP_BANKFEE – EXP_NETWORK – EXP_ROUNDING – EXP_LOSS ± VAR_CONVERSION ± VAR_CUTOFF ± VAR_RATE_HONOUR ± VAR_FX_TIMING) + reseller earnings (RESELL_REVENUE – RESELL_COST), each callable on its own and together, by project, partner, pair or month, per currency; one figure across currencies is a REPORTING_VALUE.

Notes

- **Email OTP (CR-01).** On a project with the feature on, a quote issue is an approval email with a one-time code, and the customer's acceptance is that code entered by Operations from the registered chat group: `CUSTOMER_APPROVAL` (one row per code; hash only; write-once decision; the email snapshot and the chat screenshot as evidence) and `CUSTOMER_APPROVAL_EVENT` (sends, deliveries, replies, attempts). The group's states are unchanged: Sent is pending customer approval, Accepted is customer approved. The settings sit on `PROJECT` (the switch), `SENDER` (one approver address and one registered chat group per sender, because each sender has its own chat group) and `THRESHOLD` (code window, step-up amount, tripwire window). Both tables are in Appendix B.3; Diagram 3a draws the approval record beside the quote package, and its event table is listed with it.
- **Every deal belongs to a deal group.** `DEAL_GROUP` is the client-facing instruction: the pair as the sender sees it, the sender rate, fee, reference and entry mode. A plain single-partner deal is a group of one leg; a route that needs two partners (INR → USDT at Ali, USDT → EUR at Jeton) has two forward legs; a rejected payout is unwound with a return leg.

- **Legs.** `leg_type` forward or return; `funding_source` collection, prior_leg (leg 2 of a route, no collection) or returned_settlement (a return leg). Priced once and partner fixed at quote hold per leg. Attribution is to the receiver group: set at quote on the first forward leg, inherited by leg n of a route from leg n−1 and by a return leg from its parent leg. The receiver is optional until settlement.
- Two entry modes as before: collect-first legs are born in state Collected with priced_at = conversion time; quote-first legs carry sent / valid / grace timestamps and a late decision. An estimate-first mode gives an indicative figure and prices at conversion.
- **Conversion has exactly one owner**: a deal leg or a `BALANCE_CONVERSION`. The receiver-side fee is taken at conversion. Amounts are rounded at conversion by the pair's option; a dropped fraction (`rounding_residual`, signed) stays client money in the balance and pays out when it reaches a unit, an amount added by rounding up is NPL's cost. NPL's margin is captured at conversion and is never again at risk from a return or a forward.
- **Balance conversion** is NPL converting a client's held balance (5K SGD → USDT to fund a 15K USDT payout): Finance-initiated, Management-approved, no sender, receiver or collection, same rate machinery, markup by `FEE_DECISION`. It names the receiver group it serves; `ENTITLEMENT_REATTRIBUTION` moves that group's entitlement to the new currency, and any excess over the group's entitlement is booked as an approved offset.
- **Reroute** is the only way partner or partner entity change after quote. Forwarding is forward_full or forward_and_recover_dues: NPL nets what the alternate partner owes it (`EARNINGS_RECEIVABLE`) against the funds forwarded, within the outstanding receivable, with the partner's agreement recorded. Post-collection outcome is via the own wallet or a refund to the sender. A reroute after conversion (`phase` post_conversion, NPL decision of 20 October 2026) starts from a leg in Converted: the converting partner converts the balance back to the collection currency (`return_conversion_id`, a balance conversion owned by the reroute, no markup), pays it to NPL's own wallet, and NPL forwards it to the alternate partner, which converts at its own rate of the day (`second_conversion_id`); the leg's client net is unchanged, the `rate_difference` is NPL's gain or cost (VAR_CONVERSION, Ledger Posting Design Pattern O), and Management's ratification (`ratified_by`) is needed in addition to Finance's because NPL's wallet holds client money in the meantime. Own-wallet custody cases therefore become four.
- One settlement model: `SETTLEMENT` drains one partner's holding into one or many `SETTLEMENT_LINE`s, rounded as the pair says. Under settlement_policy to_zero the balance after release is less than one unit; under hold_allowed the balance may rest at the partner within the project's exposure cap.
- A settlement line records when it was released (`released_at`), by when the rail should confirm it (`expected_confirmation_by`) and when it ended (`confirmed_at`), so the named state "sent, not yet confirmed" can be shown and aged (Lifecycle & Approvals §7; Ledger Posting Design rule 13a). The promise is discharged on release; a return or a short delivery re-raises it.
- A line paying a counterparty receiver whose group differs from the paying leg's attributed group is an offset, created by the same action, with Management approval and tighter aging. A line paying a sender_return or partner_transit receiver is never an offset: those receivers have no group, and the line debits the group of the leg it names in `deal_id`.
- **Rejected payout.** When the receiving or intermediary bank rejects a transfer, `SETTLEMENT_RETURN` records the returned amount, bank fee and reason; the line's outcome becomes returned, a reversing ledger transaction puts the money back into the client's balance and the paying leg's group entitlement, and `BANK_FEE_EVENT` books the bounce fee. The money then stays in balance for later payouts, or funds a return leg — attributed to the same group — that converts back and pays the sender's return receiver.
- **Fee decisions.** `FEE_DECISION` is a one-off, approved decision on a single subject: markup applied or waived on a return leg or balance conversion, a network-fee override on a refund, a bank-fee treatment override on one settlement. Standing rules stay in `FEE_STRUCTURE` and `FEE_OVERRIDE`.

- `CONFIRMATION` is full or short; a short confirmation opens a `SHORTFALL` topped up by a later settlement or an explicit separate payout — cash found short at hand-over included. Rounding differences no longer land here. `MARKUP_SHARE_ACCRUAL` and `PARTNER_REBATE_ACCRUAL` accrue per deal under their rules; a party_retains markup-share accrual is a receivable from the party, reconciled monthly like a rebate; rebates are reconciled monthly against the partner's statement, which covers NPLify deals only.
- **Pricing stamps.** A leg stamps the override and the base tier that priced it; a leg that crosses a tier threshold holds two `DEAL_FEE_TIER` portions and a blended fee. The deal group stamps the customer-facing fee and rate the quote printed. On a pair that needs a bank lock the leg records `rate_locked_at` and `conversion_due_date`: locked before the partner's cutoff it converts the same day, otherwise the next business day at that day's version; the sender's figures are write-once, NPL absorbs the difference as a cutoff-timing variance, and a conversion whose next-day rate breaches the margin is deferred with a reason. The client balance view shows the locked leg's converted value as pending from the lock; a void removes it. `QUOTE_PACKAGE` is the record of what was sent for a deal group — template version, channel, printed figures, `sent_at`, hash — one row per issue; a quote-first leg is Sent only when a package has sent_at, re-issues are new rows, a new price is a new deal group.
- **Invoices.** `INVOICE` is an obligation in its own currency — a vendor's EUR invoice a reseller customer pays in THB or USDT, in parts, over days. A deal group names the invoice it pays (advance, then balance); each payout line records how much of the obligation it discharged and the market snapshot used; `INVOICE_BALANCE` is the open remainder.
- **Cash and crypto parts.** A cash part carries the token (the local-currency bill whose serial identifies the agent), the agent's contact and the partner's count; the leg carries the city, which also selects the rate version. A crypto part carries the sender's declared address and the screening that admitted it, unless the project skips up-front screening.
- **Loss events.** `LOSS_EVENT` records money lost after it was verified or released — a frozen local account, cash short that cannot be recovered, a wrong-chain transfer — on either side, by any method, for a partial or full amount, with the loss split across sender, NPL, partner and client. The whole loss is booked to NPL and the client made whole in one transaction (source LossEvent; MAKEGOOD / LOSS_PAYABLE when it cannot be funded at once); Management approves the split and each share is a separate `LOSS_RECOVERY` posting; the collection part stays verified and is reversed only when the deal is unwound.

4 · Ledger & controls domain

Diagram 4. OWN_WALLET, MARKUP_SHARE_PARTY, PROJECT are defined in Diagram 1a; PARTNER_CONFIG, RECEIVER, SENDER, RECEIVER_GROUP in 1b; COLLECTION, INVOICE in 3a; CONVERSION, BALANCE_CONVERSION, REROUTE, EARNINGS_OFFSET in 3b; SETTLEMENT, SETTLEMENT_RETURN, CONFIRMATION, BANK_FEE_EVENT, MARKUP_SHARE_ACCRUAL, PARTNER_REBATE_ACCRUAL, LOSS_EVENT, LOSS_RECOVERY in 3c.

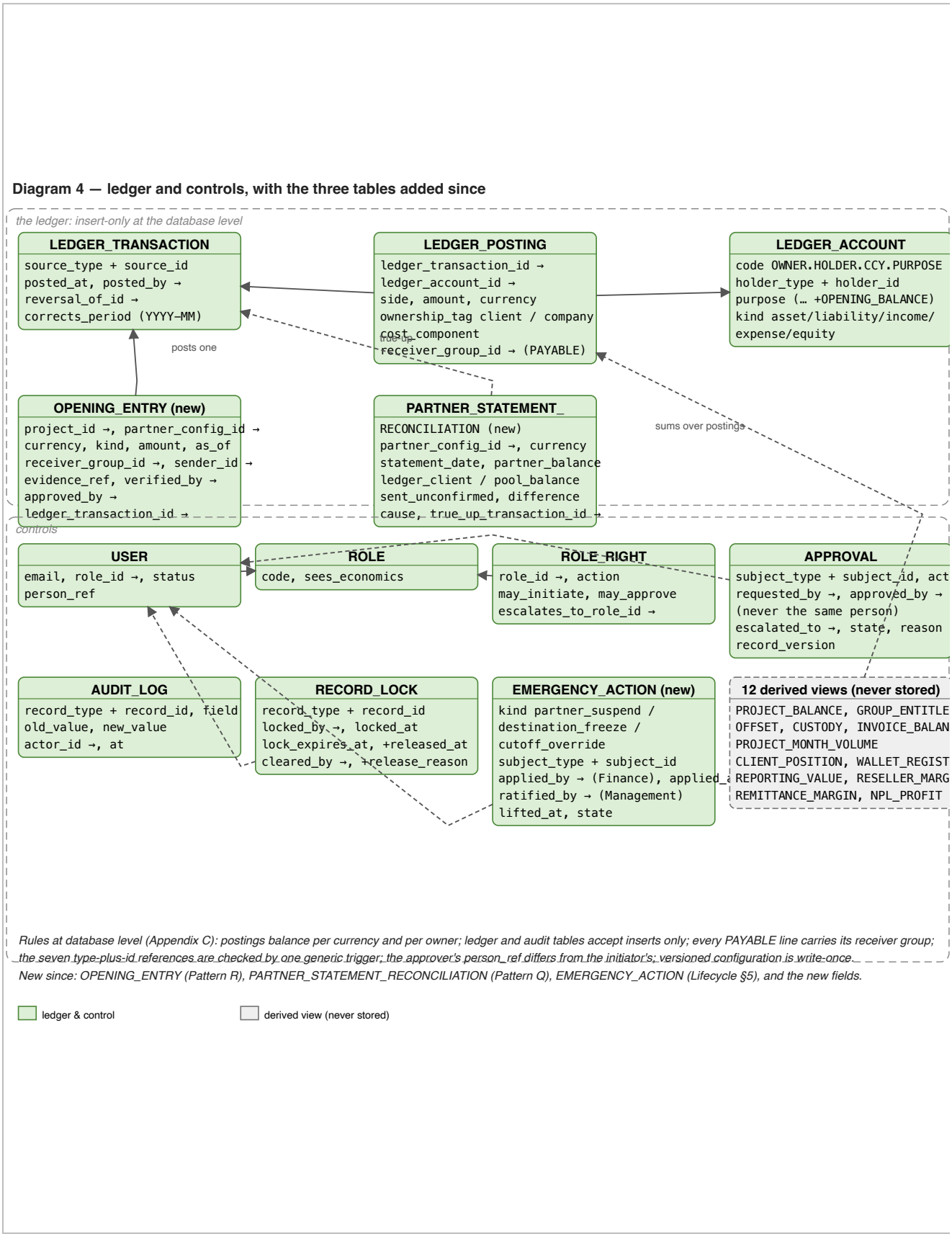


Diagram 4 — Ledger & controls

Notes

- Client due balances are held as one account per project × partner × currency (CL.<PARTNER>.<CCY>.DUE); group entitlement is a view, not an account. `holder_type` never includes a settlement rail or narrative entity.
- **Ownership moves between client and company only at named events** — margin recognition at conversion, the two sides of an earnings offset, the forex true-ups of invoice lines, fees NPL absorbs or covers, top-ups, make-wholes, make-goods and the client's loss share — and every transaction balances per owner as well as per currency. NPL never advances money to a receiver, and withholding on a reroute is not a separate event because the margin is always captured by whichever partner converts.
- Recover dues on a reroute posts own-wallet cash against the earnings receivable at the partner: company money on both sides, no income.
- Transaction sources: Collection, Conversion, BalanceConversion, Settlement, SettlementReturn, Confirmation, BankFeeEvent, Reroute, MarkupShare, Rebate, LossEvent, LossRecovery, Invoice, EarningsOffset, Adjustment, Opening. Account purposes include PAYABLE, SHARE_PAYABLE, SHARE_RECEIVABLE, REBATE_RECEIVABLE, LOSS_RECEIVABLE, MAKEGOOD / LOSS_PAYABLE, EARN_GROSS, EARN_REBATE, the four VAR_ accounts, FX_RESULT, the reseller purposes (RESELL_REVENUE, RESELL_COST, VENDOR_PAYABLE, RESALE_RECEIVABLE, REMIT_CLAIM), EXP_ROUNDING and EXP_LOSS; ADVANCE and INTRANSIT do not exist. Cost components include share, rebate, rounding, fx_timing, cutoff_timing, loss and resale.
- **Three streams NPL never mixes.** Earnings on deals post to EARN_GROSS at conversion; a partner's rebate posts to EARN_REBATE and is never in the base a client party shares; NPL's remainder from a party that holds the converted funds sits in SHARE_RECEIVABLE until the party pays it. The cost of rounding up is EXP_ROUNDING. A loss is booked to EXP_LOSS in full first and the client made whole; the partner's and the sender's agreed shares are LOSS_RECEIVABLE owed to NPL, the client's share is a partial reversal of the make-good, and what remains in EXP_LOSS is NPL's own share. Own-desk deals use the ordinary partner-coded accounts with the own-desk configuration as holder and EXP_PARTNER zero by construction.
- One system-settings area: `SYSTEM_SETTING` holds NPL-wide, Management-approved keys — the reporting currency, its staleness limit, the operating time zone; `ROLE` holds the user roles and `ROLE_RIGHT` what each may initiate and approve per controlled action, with the role added on escalation; platform-scope `CURRENCY_PAIR`s carry the conversion rates through the ordinary rate-source chain. `REPORTING_VALUE` converts any per-currency figure into the reporting currency at the latest market rate of its platform pair, as a display-only estimate that carries the rate it used. `APPROVAL`, `AUDIT_LOG` and `RECORD_LOCK` are modelled once and apply to all master data. Approver ≠ initiator on every approval; Operations never sees derived economics. Approval actions include price_quote, extend_quote_validity, refund_to_third_party, cross_group_payout, balance_conversion, recover_dues, cover_bank_fee, raise_fee_cap, enable_own_desk, approve_loss_split, screening_second_look, earnings_offset, bank_details_approve and the four Email OTP actions (ROLE_RIGHT rows). cross_group_payout never fires on a line paying a sender_return or partner_transit receiver. Wallet screening itself is an Operations action, not an approval.
- **Three tables of this edition.** `OPENING_ENTRY` is one row per opening balance entered on the opening date (Cutover Approach, Section 4; Ledger Posting Design, Pattern R), posted once, one transaction per row (source Opening): company-side balances against CO.NONE.<CCY>.OPENING_BALANCE, client-side balances between the client's jar and the project's promise, and OPENING_BALANCE is never posted to again. `PARTNER_STATEMENT_RECONCILIATION` is one row per partner configuration, currency and statement date (Pattern Q): a row for every business day of the parallel run and for every month end afterwards, a difference cleared only by a true-up transaction, never by an edit, and a row in state difference_open on the exception list. `EMERGENCY_ACTION` is the record behind the controlled action emergency_suspend (Lifecycle & Approvals, section 5): applied by Finance, in effect at once, lifted unless

Management ratifies it within one business day; ratified_by is never applied_by. `LEDGER_TRANSACTION.corrects_period` names the month a correction belongs to when it is posted in a later month (rule 11a).

- **Controls on the person and the record version.** The approver is never the initiator *on the person*: `USER.person_ref` names the natural person behind every account, and `APPROVAL.approved_by` is refused when its person_ref equals requested_by's, so a user holding two accounts cannot approve their own request (Roles & Visibility rule 5a). An approval is valid only for the record version the approver saw, so two approvers at once leave exactly one winner and a changed record is re-approved (Lifecycle & Approvals 6a). At most one open lock per record: a `RECORD_LOCK` is released on save or cancel, expires after the project's lock idle time, or is cleared by Management, and every release is written to `AUDIT_LOG`. Rules at database level are listed in Appendix C: postings balance per currency and per owner; ledger and audit tables accept inserts only; every PAYABLE line carries its receiver group; the seven type-plus-id references are checked by one generic trigger; the approver's person_ref differs from the initiator's; versioned configuration is write-once.

Appendix A · FRS reconciliation

A.1 Vocabulary

FRS term	Model entity
Client	CLIENT (the commercial relationship) — its arrangements are PROJECTs (until the FRS client mapped to PROJECT)
Project (NPL's 'Project Context': BF, BF Sub, Evo ...)	PROJECT under CLIENT — one arrangement with its own fees, partners and policies
Receiving Entity	RECEIVING_ENTITY (member of a counterparty RECEIVER, which belongs to a RECEIVER_GROUP) with its RECEIVING_ENTITY_ACCOUNTs
Sender	SENDER with SENDER_RECEIVER_ALLOW; its refund destination is a group-less sender_return RECEIVER
Partner (PT, Jeton)	PARTNER + PARTNER_CONFIG
Facilitating Entity / Wallet registry	COLLECTION_RECEIVING_ENDPOINT (kind wallet) under PARTNER_ENTITY for the partner's wallets; the FRS 'wallet registry' across all owners is the WALLET_REGISTRY view
Rate-day versions	RATE_COMPARISON + PARTNER_RATE_VERSION
Settlement	SETTLEMENT + SETTLEMENT_LINE + CONFIRMATION (+ SETTLEMENT_RETURN when rejected)
Shortfall ledger	SHORTFALL + CL.<RECEIVER>.<CCY>.SHORTFALL
PT Comms / H Comms	CO.<PARTNER>.<CCY>.POOL (earnings receivable)
Inter-entity redirection	Offset across receiver groups: a counterparty paid_receiver_id in another group than the paying leg's
Double conversion / pass-through	DEAL_GROUP with two legs (funding_source = prior_leg), hop via a group-less partner_transit RECEIVER
Commission / rebate (Jeton 0.4 %, Aquanow 0.3 %)	PARTNER_REBATE_RULE (basis pct_of_amount in payout_currency, scoped by sender or fee structure) + PARTNER_REBATE_ACCRUAL; income in EARN_REBATE
Markup shared with a customer (Raeen sub-accounts, downlines)	MARKUP_SHARE_RULE basis share_of_net_markup, party_kind client_party, settlement_mode party_retains; SHARE_RECEIVABLE
Fee by monthly volume (GDC)	FEE_TIER under FEE_STRUCTURE; DEAL_FEE_TIER portions; PROJECT_MONTH_VOLUME
Game reseller / NPL-GR	PROJECT with an own-desk PARTNER_CONFIG; OWN_WALLET as COLLECTION_RECEIVING_ENDPOINT; INVOICE paid in parts
Customer-facing calculation ('market rate', 'fee 1 %')	FEE_STRUCTURE.customer_facing_fee_basis / pct; DEAL_GROUP.customer_facing_rate
Token (cash collection / settlement)	COLLECTION.token_ref, agent_contact_ref, counted_at; SETTLEMENT_LINE.token_ref; DEAL.cash_location; PARTNER_RATE_VERSION.location
Bank cutoff (2.30 pm GMT+8)	PARTNER.bank_cutoff_time + PARTNER_CUTOFF_OVERRIDE (extensions, early closes); PARTNER_PAIR.needs_bank_lock; DEAL.rate_locked_at, cutoff_at, conversion_due_date; SETTLEMENT.cutoff_at, execution_due_date
Wallet screening	WALLET_SCREENING; COLLECTION.sending_address; COLLECTION_RECEIVING_ENDPOINT.risk_band

FRS term	Model entity
Reversal of collection; cash short	`LOSS_EVENT with its split; SHORTFALL for recoverable cash shortages`
Holiday calendar	`PARTNER.holiday_calendar (the calendar's name) + HOLIDAY rows (the dates); business-day arithmetic for due dates`
Quote package	`QUOTE_PACKAGE under DEAL_GROUP — one row per issue, hash = DEAL.quote_hash, sent_at starts validity`
Message template	`MESSAGE_TEMPLATE — versioned, global or per project, sender-facing fields only for quotes`
Settlement (FRS) — was DISBURSEMENT	`SETTLEMENT, SETTLEMENT_LINE, SETTLEMENT_RETURN; PROJECT.settlement_policy`
Markup shared with a party — was INTRODUCER / REFERRAL_*	`MARKUP_SHARE_PARTY, MARKUP_SHARE_RULE, MARKUP_SHARE_ACCRUAL; purpose SHARE_PAYABLE`
Approved destination — was SETTLEMENT_REGISTRATION	`DESTINATION_APPROVAL`
Sender pays / receiver pays; lowest and highest markup; customer-facing calculation; fee on top of market / fee inside its rate; agent rate	`sender_pays_pct, receiver_pays_pct; min_markup_pct, max_markup_pct; customer_facing_fee_basis / _pct / _rate; partner_fee_pct with fee_on_market / fee_in_rate; rate_basis market / agent_rate`
Client balance at a partner — was CLIENT_BALANCE	`PROJECT_BALANCE(project, partner, currency); the client-level roll-up is CLIENT_POSITION`
System / reporting currency	`SYSTEM_SETTING reporting_currency + platform-scope CURRENCY_PAIRs (their PAIR_RATE_SOURCE) + REPORTING_VALUE view, an estimate at the latest market rate`
User roles (Operations, Finance, Management)	`ROLE rows referenced by USER.role_id, with one ROLE_RIGHT row per role × controlled action (initiate / approve / escalate); rights per role in the Roles & Visibility Matrix`

A.2 Deviations from the FRS (classified)

FRS statement	Model	Classification
P1 — no custody	No custody except under an approved reroute or in an own-desk project (NPL-GR); OWN_WALLET + REROUTE for contingency, and the custody view as the control	Decided (answered;, 9 October 2026)
P3 — partner books never mix	Accounts per partner never net; alternate partner on reroute; post-collection transfer only if confirms	Decided (answered: reroute with recover dues; partner accounts never net)
Strict Client → Entity → Sender, one parent each	RECEIVER_GROUP → RECEIVER → RECEIVING_ENTITY → account; SENDER with a group allow-list; FRS project = one default group	Generalisation per NPL instruction
Beyond grace → re-quote always	late_collection_policy per project; FRS project keeps requote	Per NPL instruction (Sept 2026)
Sender always priced on Kraken	FEE_STRUCTURE.rate_basis; FRS project = market	Generalisation
One settlement per deal, full amount	Accumulate → settle the balance rounded as the pair says (residual carried), or hold within the exposure cap where the project allows	Generalisation
Partner fixed at quote	Same, except under REROUTE	Consistent

FRS statement	Model	Classification
Bank fee → shortfall → top-up	carry_forward is the FRS behaviour; waived / absorbed added	Per NPL instruction
Registry-only destinations	DESTINATION_APPROVAL.approval_status — FRS project has all approved	Generalisation
AQN-LT-Sub offset-settled collections	OWN_WALLET at Aquanow; pass-through is a two-leg deal group or a reroute via the own wallet; also NPL-GR's own-desk collection endpoint	Resolved

Appendix B · Data dictionary (field annotations)

Every table also carries `created_at` and `created_by`. Types: int, string, bool, decimal, datetime, date.

B.1 Configuration

PROJECT

Field	Type	Key	Notes
id	int	PK	
client_id	int	FK	the client this arrangement belongs to
name	string		
client_label	string		
quote_freshness_min	int		
quote_validity_min	int		
grace_min	int		
grace_drift_pct	decimal		
late_collection_policy	string		requote / honor_with_approval
bank_fee_treatment_default	string		waived / carry_forward / absorbed
receiver_fee_timing	string		at_conversion (default) / post_confirmation
network_fee_policy	string		absorbed / charged_back (project default; per-transaction override via FEE_DECISION). A network fee may or may not apply; when it does it is stated in USDT, when it does not it is zero and nothing posts
settlement_policy	string		to_zero / hold_allowed
shortfall_tolerance	decimal		
exposure_ceiling	decimal		cap on balance resting at a partner
status	string		
state	string		draft / validated / active · default draft; only an active project can price a deal; validated means every cross-entity rule of the Configuration Schema (section 6) passes for the project and each of its partner configuration blocks; Management activates (Configuration Schema 1a)
is_test	bool		default false; a flagged dummy twin project for scripted test deals on staging (Cutover Approach, Section 3); the clean-target extract that builds production (Cutover Approach 7) leaves the project and every row that references it behind, through its deals, settlements, approvals, evidence and postings; nothing is deleted, staging keeps them as the read-only record
archetype_code	string		nullable; the archetype used to create the project (Client Operating Procedures Part C)
cloned_from_project_id	int	FK	nullable; the source project of a clone; every copied row carries clone_confirmed_at empty until a user confirms or changes it, and activation needs them all confirmed (Configuration Schema 1a and rule 15)
email_otp_enabled	bool		default false; Email OTP, the customer's approval of each quote by a one-time code (Lifecycle 2e); on only with the three Email OTP thresholds set; each sender then needs its own approved approver address and chat group

PARTNER

Field	Type	Key	Notes
id	int	PK	
name	string		Ali, Jeton, PT

Field	Type	Key	Notes
disclosure_style	string		fee_in_rate / fee_on_market — whether the partner's fee sits inside its rate or is stated on top of market
rate_expiry_time	string		
holiday_calendar	string		name of the source calendar (e.g. ID banking holidays); the dates are HOLIDAY rows
supported_methods	string		crypto / bank / cash / local_deposit — the methods this partner offers (Ali: all four; Jeton and Aquanow: crypto, bank). Networks are implied by its wallet endpoints
status	string		
bank_cutoff_time	time		nullable: the partner's standing daily bank cutoff for pairs that rely on the banking channel (Ali and other Asian partners, 14:30); empty for Jeton and Aquanow, which bind nothing. Governs locking the rate for same-day conversion and initiating an outgoing bank settlement; per-day changes in PARTNER_CUTOFF_OVERRIDE
cutoff_timezone	string		IANA zone, e.g. Asia/Hong_Kong

RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
code	string		KRAKEN / CMC / OANDA / XE
staleness_threshold_s	int		

MARKUP_SHARE_PARTY

Field	Type	Key	Notes
id	int	PK	
name	string		
status	string		

SENDER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		Sender A, Sender B
risk_tier	string		
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)
email_otp_approver_email	string		nullable; the sender's one registered approver address (each sender has its own chat group with NPL, so the approver is per sender); protected master data: Operations enters, Finance approves, verified by a code before it is active (Roles rule 13); required to quote the sender while the project's switch is on
email_otp_chat_platform	string		nullable; the sender's registered chat group: platform
email_otp_chat_group_name	string		nullable; the sender's registered chat group: name
email_otp_chat_group_id	string		nullable; the group id where the platform gives one; the three chat columns change together, under the same approval; the row shows the current values, the audit log the history

RECEIVER_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
name	string		
is_default	bool		exactly one per project
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

RECEIVER

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
group_id	int	FK	receiver group; null unless kind = counterparty
kind	string		counterparty / sender_return / partner_transit
sender_id	int	FK	when kind = sender_return
partner_config_id	int	FK	when kind = partner_transit
name	string		the group: Receiver X, Receiver Y
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

OWN_WALLET

Client money may rest in NPL's own wallet in four cases, not three — an approved reroute before conversion, a reroute after conversion (REROUTE.phase post_conversion), a collection bridge within custody_max_hours, and an own-desk leg; role reroute_contingency serves both reroute phases and the CUSTODY view names the reroute and its phase for every resting balance.

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
provider	string		e.g. Aquanow
account_label	string		e.g. LT Sub
role	string		reroute_contingency / own_desk / collection_bridge — LT Sub at Aquanow is the NPL-GR own desk; NPL's ERC-20 wallet is a bridge for partners without that network
networks	string		networks the wallet receives on (ERC-20 where Ali cannot)
network	string		
address	string		
signer_arrangement	string		hardware / multisig
custody_max_hours	int		

THRESHOLD

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	

Field	Type	Key	Notes
kind	string		max_amount / settlement_limit / daily_slots / rate_variation_rel / custom; rate_min_observations, rate_fallback_spread_abs, rate_abs_tolerance (Calculation Specification 2a); approval_time_limit (one row per approval stage, Lifecycle 8a); cutoff_warning_min; lock_idle_minutes (default 15; Lifecycle 6a); email_otp_code_window_min (default 1440), email_otp_stepup_amount (in the reporting currency), email_otp_tripwire_min (default 240) — the Email OTP windows (Lifecycle 2e); route_unused_days (dormant routes)
currency	string		
value	decimal		
condition_expr	string		custom conditions
escalation	string		finance / management
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

CURRENCY_PAIR

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	null when scope = platform
scope	string		project / platform — a platform pair (currency → reporting currency) belongs to no project, is managed under system settings, is priced through PAIR_RATE_SOURCE like any pair and is never quoted
from_currency	string		
to_currency	string		
quote_direction	string		out_per_in / in_per_out
rate_precision	int		
amount_rounding	string		exact / truncate_unit / nearest_unit / round_up_unit / nearest_n / round_up_n (exact is the default for bank transfers — cents go out; the unit options apply where the pair says so, in practice to cash hand-overs)
cash_rounding	string		truncate_n / nearest_n / round_up_n — how the collected cash amount is rounded to cash_rounding_unit only)
cash_rounding_unit	int		100 / 500 / 1000 by currency — the unit the collected cash amount is rounded to; AED cash 100, USD cash in Dubai 10 (NPL, 9 Oct)
status	string		
rounding_unit	decimal		required for nearest_n / round_up_n (1,000 for THB, INR, JPY payouts)
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

PARTNER_CONFIG

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_id	int	FK	
applicable_for_reroute	bool		
status	string		
is_own_desk	bool		default false; NPL converts in its own wallet for this project (NPL-GR); enabling needs Management approval

Field	Type	Key	Notes
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

COLLECTION_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
label	string		sender's own account or a client's account
bank_account_details_id	int	FK	nullable; the source account when it is known
wallet_network	string		nullable; the network of a source wallet (TRC-20, ERC-20, …); set together with wallet_address
wallet_address	string		nullable; the address the funds come from; a row carries a bank-details row or a wallet network and address, exactly one of the two; never a payout destination

SENDER_RECEIVER_ALLOW

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
receiver_group_id	int	FK	sendable_to
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

RECEIVING_ENTITY

Field	Type	Key	Notes
id	int	PK	
receiver_id	int	FK	
name	string		Entity X1, Entity X2
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

PAIR_RATE_SOURCE

Field	Type	Key	Notes
id	int	PK	
currency_pair_id	int	FK	
rate_source_id	int	FK	
priority	int		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

PARTNER_PAIR

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	

Field	Type	Key	Notes
currency_pair_id	int	FK	
partner_pricing	string		fee_on_market / fee_in_rate
partner_fee_pct	decimal		known only for fee_on_market
status	string		active / inactive / dormant; a dormant pair prices nothing and accepts no collection or settlement line; a route unused for the project's route_unused_days threshold is put on the quarterly review to be set dormant or kept (Scenario Catalogue, dormant products)
needs_bank_lock	bool		the pair relies on the banking channel, so the partner's cutoff governs its rate lock and its settlement (Evo pairs into EUR; USDT → USD / SGD at Ali); crypto-only pairs do not
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

SETTLEMENT_SENDING_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
narrative_name	string		what the receiver sees
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

SETTLEMENT_RAIL

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		bank or EMI executing the transfer
bank_account_details_id	int	FK	the rail's own account
expected_bank_fee	decimal		
currency	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

FEE_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
partner_pair_id	int	FK	
pct	decimal		
effective_from	datetime		
approved_by	int	FK	
sender_pays_pct	decimal		optional; with receiver_pays_pct replaces the structure's proportion
receiver_pays_pct	decimal		optional; both blank → structure's proportion; both set → sum = pct

Field	Type	Key	Notes
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

RECEIVING_ENTITY_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
receiving_entity_id	int	FK	
currency	string		
rail	string		SWIFT / SEPA / TRC-20 / ERC-20 / …
account_kind	string		bank / wallet
bank_account_details_id	int	FK	when account_kind = bank; the current BANK_ACCOUNT_DETAILS version
network	string		when account_kind = wallet
address	string		when account_kind = wallet; checksum-validated per network
status	string		
usage	string		standing / single_use — a local account the receiver gives for one payout
reserved_for_deal_id	int	FK	nullable; the one deal a single_use account serves
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

DESTINATION_APPROVAL

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
receiving_entity_account_id	int	FK	
approval_status	string		pending_partner / approved / suspended
bank_account_details_id	int	FK	the bank details as registered with this partner, when the account is a bank account
network	string		as registered with this partner; only when the account's kind is wallet
address	string		as registered with this partner; only when the account's kind is wallet
approved_at	datetime		
wallet_screening_id	int	FK	nullable; the receiver-side screening of a wallet account
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

FEE_STRUCTURE

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_basis	string		market / agent_rate
mode	string		fixed / variable
pct	decimal		
sender_pays_pct	decimal		
receiver_pays_pct	decimal		

Field	Type	Key	Notes
min_markup_pct	decimal		
max_markup_pct	decimal		
min_margin_pct	decimal		
variance_threshold_rel	decimal		
quoting_rate_policy	string		worst_applicable / designated / best_applicable
effective_from	datetime		
customer_facing_fee_basis	string		actual / sender_pays_on_agent_rate / fixed
customer_facing_fee_pct	decimal		when customer_facing_fee_basis = fixed (BF via Ali shows 1 %)
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

MARKUP_SHARE_RULE

Field	Type	Key	Notes
id	int	PK	
markup_share_party_id	int	FK	
project_id	int	FK	
partner_pair_id	int	FK	optional
basis	string		share_of_earnings / fixed_pct / share_of_net_markup (base = fee parts – partner cost on the leg, on amount × market; rebates never in the base)
pct	decimal		
effective_from	datetime		
fee_structure_id	int	FK	optional scope: only deals priced on this structure (Raeen sub-account 50 %, Raeen direct downlines 75 %)
party_kind	string		markup-share party / client_party
settlement_mode	string		npl_pays / party_retains — the party holds the converted funds and pays NPL the remainder monthly
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

PARTNER_REBATE_RULE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
effective_from	datetime		
basis	string		pct_of_amount — the rebate is a percentage of the amount converted (Jeton 0.4 % of amount on a 1 % or 1.25 % fee, Novi 0.3 %, Aquanow 0.3 %)
pct	decimal		the rebate in the chosen basis
fee_structure_id	int	FK	optional scope
sender_id	int	FK	optional scope (Novi)
payout_currency	string		the currency the partner pays the rebate in and the accrual is expressed in — the amount currency
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

PARTNER_ENTITY

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
name	string		the partner's legal vehicle: PT-tour, PT Global Inc
status	string		

COLLECTION_RECEIVING_ENDPOINT

Field	Type	Key	Notes
id	int	PK	
partner_entity_id	int	FK	the vehicle whose name is on the account or wallet
kind	string		bank / wallet
currency	string		
rail	string		nullable: a bank endpoint needs it to go active; optional for wallets, must match network when set
bank_account_details_id	int	FK	when kind = bank; the current BANK_ACCOUNT_DETAILS version
network	string		TRC-20 / ERC-20 / …; when kind = wallet
address	string		when kind = wallet; checksum-validated per network
status	string		
own_wallet_id	int	FK	nullable; set when the endpoint is NPL's own wallet — under an own-desk configuration or as a collection bridge for a partner that lacks the network or for risk isolation; the endpoint still belongs to the leg's partner configuration
risk_band	string		standard / elevated — which screening decisions may land here
usage	string		standing / single_use (ad hoc local deposit accounts)
reserved_for_deal_id	int	FK	nullable; the one deal a single_use endpoint serves
expected_amount	decimal		nullable; the amount a single_use endpoint for
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

FEE_TIER

Field	Type	Key	Notes
id	int	PK	
fee_structure_id	int	FK	
min_monthly_volume	decimal		threshold in volume_currency, per calendar month
volume_currency	string		
pct	decimal		
sender_pays_pct	decimal		optional
receiver_pays_pct	decimal		optional
effective_from	datetime		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

WALLET_SCREENING

A screening of one wallet address on one day — a sender's declared sending wallet before a collection, or a receiver's wallet before a settlement — with the result, the decision and who made it. Operations records it as part of transaction coordination. The decision picks the endpoint: accept_alternate lands only on an elevated-band endpoint. NPL screens; the partners decide: a clear

report under the provider's strict rulebook is Operations' decision, anything else is Management's second look, and no role has a right that lifts a partner's hold.

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
subject_kind	string		sender_wallet / receiver_wallet
sender_id	int	FK	when subject_kind = sender_wallet
receiving_entity_account_id	int	FK	when subject_kind = receiver_wallet
network	string		
address	string		
provider_ref	string		screening tool reference
result	string		standard / elevated / unacceptable
decision	string		accept / accept_alternate / reject; write-once for every role and channel — a reject is lifted only by a new screening row decided by Management on a second look; a partner's own hold or freeze is recorded as the event it is and never overridden at NPL (Roles rule 11)
screened_by	int	FK	Operations
screened_at	datetime		
evidence_ref	string		Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
api_response	json		the provider's report as returned, in one field: the rulebook outcome, the risk score and band, the categories against their thresholds, the exposure by channel; no separate columns
decided_by	int	FK	USER: Operations on a report that is clear under the provider's strict rulebook; Management on a second look, never the person who screened
decided_at	datetime		
reason	string		the decision's reason; mandatory on a second look

PARTNER_CUTOFF_OVERRIDE

Field	Type	Key	Notes
id	int	PK	
partner_id	int	FK	
applies_on	date		one day; null for a weekday rule
weekday	string		recurring rule (Friday); null for a one-day override
cutoff_time	time		the cutoff that applies instead
kind	string		extension / early_close
reason	string		urgent request / pre-holiday / partner notice
partner_agreement_ref	string		the partner agreed (extensions)
recorded_by	int	FK	Finance

HOLIDAY

Field	Type	Key	Notes
id	int	PK	
partner_id	int	FK	
date	date		
name	string		Idul Fitri, Boxing Day

Field	Type	Key	Notes
scope	string		bank_channel / partner
source	string		partner notice / public calendar

MESSAGE_TEMPLATE

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	null = global default; a project row overrides it
code	string		quote / payment_instructions / confirmation_request / settlement_advice / cash_token / email_otp_quote (the Email OTP approval email: the quote's sender-facing figures, the code, its expiry and the partial destination)
version	int		
channel	string		email / chat / portal / pdf
language	string		
body_ref	string		external content store Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
printed_fields	string		the model fields the template may print; sender-facing only for quote
effective_from	datetime		
approved_by	int	FK	Management
status	string		
clone_confirmed_at	datetime		nullable; set when a user confirms or changes a row copied by a clone; a cloned project activates only when no copied row is left unconfirmed (Configuration Schema rule 15)

CLIENT

Field	Type	Key	Notes
id	int	PK	
name	string		Betfair, Evo, 66 Group, GDC, NPL-GR
category	string		NPL's own grouping (Project Context tab: BF, Evo, Game Reseller ...)
legal_entity_ref	string		contracting entity, external store
kyc_ref	string		external store
relationship_owner	int	FK	USER
exposure_ceiling	decimal		optional cap on Σ balance resting at partners across the client's projects, stated in the reporting currency and compared through REPORTING_VALUE at the latest market rate
status	string		

SYSTEM_SETTING

Field	Type	Key	Notes
id	int	PK	
key	string		reporting_currency / reporting_rate_max_age_min / operating_timezone / staging_snapshot_retention_days (how long the read-only staging snapshot with the test projects' audit rows is kept after the switch) / ... (the reporting rate source is the platform pair's PAIR_RATE_SOURCE, not a key)
value	string		
effective_from	datetime		
approved_by	int	FK	Management

BANK_DETAILS_TEMPLATE

Configuration: what a bank account on a given rail must carry. One row per rail × field, in display order: SWIFT (account number or IBAN, BIC, bank address, intermediary bank), SEPA (IBAN, BIC), UK Faster Payments (sort code, account number), a local rail (bank code, account number). The template drives the entry form, the validation on save (IBAN check digits, BIC length, sort-code pattern) and the card that shows the details in full; wallets are not templated (network and address stay on the account row). Versioned by effective date like every configuration table; Finance proposes, Management approves.

Field	Type	Key	Notes
id	int	PK	
rail	string		SWIFT / SEPA / UK_FPS / local:<country> ...; one template per rail
position	int		display order within the rail
field_key	string		iban, bic, account_number, sort_code, bank_code, bank_address, intermediary_bic ...
label	string		as shown on the form and the card
required	bool		
format_rule	string		named check or pattern: iban_check_digits, bic_length, uk_sort_code, digits(n) ...
example	string		shown on the form
effective_from	date		
status	string		

BANK_ACCOUNT_DETAILS

The bank details of one account, in the database: a receiving-entity account, a partner's collection endpoint, a settlement rail, the details a partner holds under a destination approval, or a sender's source account. Versioned: a change is a new row that names the row it supersedes (supersedes_id); the old row is kept and is what any settlement line released before the change points at through its account. Common fields here; the rail-specific values (IBAN, BIC, sort code, bank code ...) are stored under rail_fields, keyed by the field_key of the rail's BANK_DETAILS_TEMPLATE rows and validated against them on save. Entered by Operations, approved by Finance or Management (controlled action bank_details_approve); read in full by Operations, Finance and Management, with every full read written to AUDIT_LOG. The evidence (bank letter, KYC, legal document) stays a file under the restricted prefix (evidence_ref).

Field	Type	Key	Notes
id	int	PK	
supersedes_id	int	FK	nullable; the BANK_ACCOUNT_DETAILS row this version replaces
holder_name	string		the account holder as the bank knows it
bank_name	string		
country	string		ISO 3166 alpha-2
currency	string		
rail	string		names the BANK_DETAILS_TEMPLATE rows that apply
rail_fields	json		the rail-specific values, keyed by the template's field_key; validated by format_rule on save
evidence_ref	string		bank letter or confirmation, restricted prefix of the file store Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
entered_by	int	FK	Operations user
entered_at	datetime		
status	string		

B.2 Rates

MARKET_RATE

Field	Type	Key	Notes
id	int	PK	
rate_source_id	int	FK	
currency_pair_id	int	FK	

Field	Type	Key	Notes
rate	decimal		
captured_at	datetime		

RATE_COMPARISON

Field	Type	Key	Notes
id	int	PK	
partner_pair_id	int	FK	
rate_date	date		
avg_spread_2w	decimal		
avg_spread_4w	decimal		
status	string		open / closed
created_at	datetime		created lazily by the first deal of the day

PARTNER_RATE_VERSION

Field	Type	Key	Notes
id	int	PK	
rate_comparison_id	int	FK	
version_no	int		
rate	decimal		normalised to CURRENCY_PAIR.quote_direction; derived from quoted_rate
market_rate_id	int	FK	
spread_pct	decimal		
status	string		active / usable / flagged / breached / approved / rejected; a flagged version (outside the absolute tolerance, either direction) prices nothing and instructs nothing until Finance confirms it with a reason or rejects it; the last usable version keeps serving meanwhile (Calculation Specification 2a)
confirmed_by	int	FK	
approval_reason	string		
expires_at	datetime		
quoted_rate	decimal		as the partner stated it (Ali: 1.1738 EUR→USDT style)
quoted_direction	string		out_per_in / in_per_out; rate = quoted_rate, or 1 ÷ quoted_rate at the pair's precision when the directions differ — derived, never typed
location	string		nullable; required on cash pairs — INR cash in Delhi and in Mumbai are two versions on one day

B.3 Deal, collection, conversion, reroute, balance conversion

DEAL_GROUP

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	entitled group
client_ref	string		D-YYMMDD-seq, what the sender sees
from_currency	string		
to_currency	string		
sender_rate	decimal		

Field	Type	Key	Notes
fee_structure_id	int	FK	
amount_in	decimal		
target_out	decimal		
entry_mode	string		quote_first / collect_first / estimate_first
state	string		
quoted_at	datetime		
customer_facing_fee_pct	decimal		stamped at quote
customer_facing_rate	decimal		= sender_rate ÷ (1 – customer_facing_fee_pct); the 'market rate' the quote prints
invoice_id	int	FK	nullable; the resale invoice this group collects (NPL-GR) — the obligation the deal pays
invoice_amount	decimal		nullable; the part of the resale invoice this group collects, in the invoice currency — an advance, then the balance; Σ over the invoice's groups ≤ its amount
invoice_rate	decimal		nullable; payout currency per invoice currency at pricing (same direction as SETTLEMENT_LINE.discharge_rate): the stored base for the amount reserved per instalment and for the forex true-up
opening_source_ref	string		nullable; NPL's remittance-log transaction number for a group entered at the opening date; never edited afterwards (Cutover Approach 4, step 6)

DEAL

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	every deal belongs to a group
leg_no	int		
leg_type	string		forward / return
funding_source	string		collection / returned_settlement / prior_leg
funding_ref_id	int		SETTLEMENT_RETURN id or prior DEAL id
project_id	int	FK	
sender_id	int	FK	
receiver_group_id	int	FK	attribution
receiver_id	int	FK	optional until settlement
partner_pair_id	int	FK	
partner_entity_id	int	FK	
amount_mode	string		target_out / fixed_in — fixed_in takes the fee off the principal, target_out adds it as a markup on the target
collection_method	string		crypto / bank / cash
cash_instructions	string		free text for the hand-over; the city is cash_location, the token is on the collection
fee_structure_id	int	FK	
market_rate_id	int	FK	the market snapshot the leg on; equals the earlier leg's when rate_reused_from_deal_id is set
partner_rate_version_id	int	FK	
sender_rate	decimal		
amount_in	decimal		
gross_out	decimal		

Field	Type	Key	Notes
client_net	decimal		
fee_sender_part	decimal		
fee_receiver_part	decimal		
priced_at	datetime		quote or conversion
sent_at	datetime		null on conversion-priced legs; = QUOTE_PACKAGE.sent_at of the package in force
valid_until	datetime		end of validity as quoted; effective_valid_until is the one in force after extensions
grace_until	datetime		
effective_valid_until	datetime		the validity in force: valid_until as quoted, moved by each approved QUOTE_VALIDITY_EXTENSION (any number, no ceiling); grace_until follows it
rate_reused_from_deal_id	int	FK	nullable; this leg on the market rate and partner rate version of an earlier, expired leg at the sender's request — an approved override of the automated rate; the difference against the day's rate posts to VAR_RATE_HONOUR with component variance (the account named by, note corrected)
rate_override_reason	string		why the earlier rate
rate_override_approved_by	int	FK	Finance
late_decision	string		none / honored / requoted
state	string		
quote_hash	string		hash of the quote package in force for the group (QUOTE_PACKAGE.content_hash)
fee_override_id	int	FK	nullable; the override that priced this leg
fee_tier_id	int	FK	nullable; base tier at quote
tier_volume_basis	decimal		project month-to-date volume that justified the tier, stamped at quote
rate_locked_at	datetime		when the partner confirmed the rate; before the cutoff → converts the same day, after → next business day
conversion_due_date	date		the lock date when rate_locked_at precedes cutoff_at, else the next business day — weekends in the partner's cutoff_timezone and the partner's HOLIDAY rows skipped; Converting may not start earlier
conversion_deferred_reason	string		set when the due date is pushed because the next day's rate breaches the minimum margin (ask for a better rate, or wait a day)
cash_location	string		city of the cash hand-over; required on cash legs — rates and agents differ by city
cutoff_at	datetime		the effective cutoff that applied on the lock day (standing time or override), stamped with rate_locked_at
hold_kind	string		none / partner_hold / screening_hold; while set, the server refuses every money-moving write on the leg: conversion, balance conversion, settlement line, refund, reroute forward, return (Lifecycle 2c)
hold_ref	string		nullable; the row that holds the leg, as type and id: an EMERGENCY_ACTION, REROUTE (compliance hold), LOSS_EVENT (account frozen) or WALLET_SCREENING row
opening_entry_id	int	FK	nullable; for a deal in flight on the opening date, the OPENING_ENTRY row that holds its money, so the engine posts nothing for the steps recorded as completed and only the next step posts (Cutover Approach 4)

COLLECTION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
collection_receiving_endpoint_id	int	FK	the partner endpoint paid into; null for cash
collection_sending_entity_id	int	FK	
amount	decimal		
evidence_ref	string		hash / slip / receipt Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
state	string		pending / verified / reversed; cash: token_issued → handed_over → counted → verified.: after a loss the part stays verified — NPL's funds stand in — and is reversed only when Management unwinds the deal
verified_by	int	FK	
verified_at	datetime		
sending_address	string		crypto: the sender's declared wallet
wallet_screening_id	int	FK	nullable; the screening that admitted the sending wallet
screening_skipped	bool		projects that do not screen every sender up front (Evo)
token_ref	string		cash: serial of the token (local-currency bill) that identifies the agent; the partner may replace the agent and the token before hand-over — the collection holds the current token, the audit log the previous one; the rate locked at quote is unchanged Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
agent_contact_ref	string		cash: the receiving agent's contact, external store
counted_at	datetime		cash: the partner counted and confirmed the funds

CONVERSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	null when owned by a balance conversion
balance_conversion_id	int	FK	null when owned by a deal leg
partner_rate_version_id	int	FK	
calc_mode	string		npl_calculates / partner_calculates
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		booked figure, rounded as the pair says
rounding_residual	decimal		signed: positive = fraction dropped, stays client money; negative = amount added by rounding up, NPL's cost posted to EXP_ROUNDING
fee_receiver_part	decimal		taken at conversion
variance	decimal		
residual_in	decimal		the margin kept unconverted in the collection currency (NPL's practice: earnings are taken out of money not yet converted); EARN_GROSS is then recognised in that currency and rests in NPL's pool at the partner in that currency
basis_variance	decimal		
state	string		

Field	Type	Key	Notes
approved_by	int	FK	
instruction_ref	string		the instruction's own reference, sent to the partner and used as the idempotency key; one open instruction per promise (Lifecycle 2d)
ack_status	string		instructed / acknowledged / rejected / unknown; the partner's acknowledgement is the first point of no return and the execution the second; unknown means no acknowledgement by the time-out and stays so until the partner's records say executed or not (Lifecycle 2d)
ack_ref	string		nullable; the partner's reference for the acknowledgement, the execution or the rejection
retry_of	int	FK	nullable; the instruction this one re-issues after the first as rejected on the partner's written confirmation; Finance authorises the one retry (Lifecycle 2d, rule 3)

REROUTE

"Reroute is the only way partner or partner entity change after quote" now holds for a converted leg too; a post_conversion reroute leaves the leg's pricing stamp and client net untouched and moves only where the balance rests.

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
trigger	string		partner_unavailable / limit_reached / holiday (a HOLIDAY row of scope partner on the day) / compliance_hold / other
phase	string		pre_collection / post_collection / post_conversion (a post_conversion reroute starts from a leg in Converted — the converting partner converts the balance back to the collection currency, pays it to NPL's own wallet, NPL forwards it to the alternate partner, which converts at its own rate of the day)
outcome	string		via_own_wallet / refund_sender; for phase post_conversion only via_own_wallet
own_wallet_id	int	FK	
alternate_partner_config_id	int	FK	
forwarding_policy	string		forward_full / forward_and_recover_dues
recovered_amount	decimal		≤ outstanding receivable at the alternate partner
partner_agreement_ref	string		the partner agreed the net amount
reason	string		
requested_by	int	FK	
approved_by	int	FK	
return_conversion_id	int	FK	nullable; BALANCE_CONVERSION — the conversion back to the collection currency at the first partner, owned by this reroute, no markup; set only when phase = post_conversion
second_conversion_id	int	FK	nullable; CONVERSION — the conversion at the alternate partner that completes the reroute; the leg's client net is unchanged and the difference between what the alternate partner delivers, less its stated cost, and the client net posts to VAR_CONVERSION (Ledger Posting Design, Pattern O)
rate_difference	decimal		nullable; the signed difference in the payout currency, NPL's gain or cost; equals the VAR_CONVERSION line of the second conversion
ratified_by	int	FK	USER, Management — a post_conversion reroute needs Management approval in addition to Finance's, because NPL's wallet holds client money in the meantime

Field	Type	Key	Notes
instruction_ref	string		the instruction's own reference, sent to the partner and used as the idempotency key; one open instruction per promise (Lifecycle 2d)
ack_status	string		instructed / acknowledged / rejected / unknown; the partner's acknowledgement is the first point of no return and the execution the second; unknown means no acknowledgement by the time-out and stays so until the partner's records say executed or not (Lifecycle 2d)
ack_ref	string		nullable; the partner's reference for the acknowledgement, the execution or the rejection
retry_of	int	FK	nullable; the instruction this one re-issues after the first as rejected on the partner's written confirmation; Finance authorises the one retry (Lifecycle 2d, rule 3)

SENDER_CREDIT

Field	Type	Key	Notes
id	int	PK	
sender_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		open / applied

EARNINGS_RECEIVABLE

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		outstanding / recovered
recovered_by_deal_id	int	FK	
recovered_by_offset_id	int	FK	nullable; the EARNINGS_OFFSET that moved this margin to another partner’s unconverted balance
recovered_at	datetime		

BALANCE_CONVERSION

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
receiver_group_id	int	FK	group served; chosen split
from_currency	string		
to_currency	string		
partner_pair_id	int	FK	
partner_rate_version_id	int	FK	
market_rate_id	int	FK	
amount_in	decimal		
expected_out	decimal		
actual_out	decimal		
variance	decimal		

Field	Type	Key	Notes
rounding_residual	decimal		
state	string		
requested_by	int	FK	Finance
approved_by	int	FK	Management
converted_at	datetime		
rate_locked_at	datetime		
conversion_due_date	date		banking-ready balances (USD, SGD) wait for the next business morning
instruction_ref	string		the instruction's own reference, sent to the partner and used as the idempotency key; one open instruction per promise (Lifecycle 2d)
ack_status	string		instructed / acknowledged / rejected / unknown; the partner's acknowledgement is the first point of no return and the execution the second; unknown means no acknowledgement by the time-out and stays so until the partner's records say executed or not (Lifecycle 2d)
ack_ref	string		nullable; the partner's reference for the acknowledgement, the execution or the rejection
retry_of	int	FK	nullable; the instruction this one re-issues after the first as rejected on the partner's written confirmation; Finance authorises the one retry (Lifecycle 2d, rule 3)

ENTITLEMENT_REATTRIBUTION

Field	Type	Key	Notes
id	int	PK	
balance_conversion_id	int	FK	
receiver_group_id	int	FK	
amount_from	decimal		
amount_to	decimal		

DEAL_FEE_TIER

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
fee_tier_id	int	FK	
amount_in_portion	decimal		Σ over a leg = amount_in
pct	decimal		the tier’s pct at quote

INVOICE

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
kind	string		vendor (the vendor’s invoice to NPL-GR at the preferential price) / resale (NPL-GR’s invoice to its customer at retail price)
vendor_invoice_id	int	FK	on a resale invoice: the vendor invoice it marks up (self-reference); RESELLER_MARGIN pairs the two through it
receiver_id	int	FK	the vendor owed, on a vendor invoice
sender_id	int	FK	the customer invoiced, on a resale invoice
currency	string		
amount	decimal		

Field	Type	Key	Notes
reference	string		
issued_at	datetime		
status	string		open / settled / void

QUOTE_PACKAGE

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	
issue_no	int		1, 2, … per deal group
message_template_id	int	FK	the template version that rendered it
channel	string		email / chat / portal / pdf
rendered_ref	string		external document store Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
content_hash	string		= DEAL.quote_hash of the legs it prices
printed_source_rate	decimal		= DEAL_GROUP.customer_facing_rate
printed_fee_pct	decimal		= DEAL_GROUP.customer_facing_fee_pct
printed_amount_out	decimal		
valid_until	datetime		= the leg's valid_until
sent_at	datetime		marks the package sent; starts validity
sent_by	int	FK	Operations
superseded_by_id	int	FK	the re-issue that replaced this one, if any

QUOTE_VALIDITY_EXTENSION

Field	Type	Key	Notes
id	int	PK	
deal_id	int	FK	
extension_no	int		1, 2, … per leg
requested_at	datetime		
from_valid_until	datetime		
to_valid_until	datetime		
reason	string		the sender's request
approved_by	int	FK	Finance; Management past the cutoff or the rate version's expiry
approval_id	int	FK	

EARNINGS_OFFSET

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
from_partner_config_id	int	FK	where NPL's converted margin rests
from_currency	string		
from_amount	decimal		NPL's margin handed to the client there
to_partner_config_id	int	FK	where the client's unconverted money is
to_currency	string		
to_amount	decimal		= from_amount ÷ the rate used, taken from the client's unconverted balance
partner_rate_version_id	int	FK	the to-partner's rate of the day (NPL, 11 Oct: the partner's rate, not market)

Field	Type	Key	Notes
market_rate_id	int	FK	the market snapshot of the day, for reference
requested_by	int	FK	Finance
approved_by	int	FK	Management
state	string		requested / approved / posted
posted_at	datetime		

CUSTOMER_APPROVAL

Email OTP: one row per one-time code sent for a quote issue; the customer's approval of the order as a recorded, attributable step (Lifecycle 2e). The code is stored only as a hash; the decision is write-once; one open code per issue; no collection without the approval of the current issue.

Field	Type	Key	Notes
id	int	PK	
deal_group_id	int	FK	the group whose quote is approved
quote_package_id	int	FK	the quote issue the code is bound to; a re-issue voids the row
code_ref	string		short public reference printed in the email, for support
code_hash	string		the one-time code, hashed; never the code itself; in no screen, export or API
approver_email	string		the address the email went to, as it issue; a change of the sender's address voids the row
issued_at	datetime		
issued_by	int	FK	USER (Operations)
expires_at	datetime		issued_at + the project's code window
stepup_required	bool		true at or above the step-up amount; the paste must then carry the amount
amount_expected	decimal		the order amount the paste must match when stepup_required
partial_destination	string		the partial identifier exactly as printed; empty for cash
snapshot_ref	string		the stored email, content-addressed (write-once)
status	string		issued / approved / rejected / expired / voided / locked
attempts	int		failed validations; five lock the code
decided_at	datetime		nullable
decided_by	int	FK	USER who entered the code or recorded the rejection
evidence_ref	string		the chat screenshot from the registered group (write-once)
reject_reason	string		verbatim, when rejected
reject_channel	string		email_reply / chat

CUSTOMER_APPROVAL_EVENT

The trail under a customer approval: every send, delivery event, bounce, reply, validation attempt, lock, void and tripwire alert, with its time and, for inbound mail, the sender address and the text; insert-only. Nothing that arrives by email ever approves.

Field	Type	Key	Notes
id	int	PK	
customer_approval_id	int	FK	
kind	string		sent / delivered / bounced / failed / reply / attempt / locked / voided / expired / tripwire
at	datetime		
actor_id	int	FK	USER, nullable (system events)
from_address	string		inbound mail only; matched against the registered address
matched_approver	bool		inbound mail only

Field	Type	Key	Notes
body_ref	string		inbound text stored, content-addressed
message_id	string		the email service's id for a send or a reply

B.4 Settlement, return, confirmation, fees, shares, rebates, losses and recoveries

SETTLEMENT

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	
settlement_sending_entity_id	int	FK	
settlement_rail_id	int	FK	
currency	string		
total	decimal		rounded as the pair says
bank_fee_treatment	string		
state	string		
prepared_by	int	FK	
approved_by	int	FK	
escalated_to	int	FK	
released_at	datetime		
cutoff_at	datetime		the effective cutoff of the approval day for a bank rail (standing time or override)
execution_due_date	date		the approval date when approved before cutoff_at, else the next business day (weekends and HOLIDAY rows skipped); a bank transfer is initiated no earlier

SETTLEMENT_LINE

Field	Type	Key	Notes
id	int	PK	
settlement_id	int	FK	
deal_id	int	FK	nullable; required when the paid receiver is group-less, optional on counterparty lines
destination_approval_id	int	FK	
paid_receiver_id	int	FK	
amount	decimal		
receiver_fee_applied	decimal		exceptional; fee is normally taken at conversion
reference	string		entity invoice format
outcome	string		sent / confirmed / short / returned — sent from the partner's acknowledgement (ack_status acknowledged) until the receiver's confirmation; short when the CONFIRMATION has result short and a SHORTFALL row re-raises the missing part; every released line ends confirmed, short or returned
invoice_id	int	FK	nullable; the invoice this line discharges — the vendor invoice for a vendor line, the resale invoice for the margin line to NPL-GR's own wallet
obligation_discharged	decimal		in the invoice currency; reserved = obligation_discharged × DEAL_GROUP.invoice_rate, paid = obligation_discharged × discharge_rate; reserved − paid is the forex true-up posted to VAR_FX_TIMING against NPL's pool

Field	Type	Key	Notes
discharge_rate	decimal		payout currency per invoice currency at payment
discharge_market_rate_id	int	FK	nullable; the market snapshot used
token_ref	string		cash settlement: the receiver's token, provided up front and relayed NPL → partner → agent Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
evidence_ref	string		cash settlement: the agent's photo of the receiver's token after hand-over, relayed partner → NPL → receiver; the receiver's CONFIRMATION closes the loop Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
released_at	datetime		when the partner released the money (outcome becomes sent); the promise is discharged at this moment
expected_confirmation_by	datetime		released_at plus the rail's expected time (SETTLEMENT_RAIL); a line still sent after this time is on the exception list
confirmed_at	datetime		nullable; set when the line ends confirmed, short or returned, from the CONFIRMATION or SETTLEMENT_RETURN row
instruction_ref	string		the instruction's own reference, sent to the partner and used as the idempotency key; one open instruction per promise (Lifecycle 2d)
ack_status	string		instructed / acknowledged / rejected / unknown; the partner's acknowledgement is the first point of no return and the execution the second; unknown means no acknowledgement by the time-out and stays so until the partner's records say executed or not (Lifecycle 2d)
ack_ref	string		nullable; the partner's reference for the acknowledgement, the execution or the rejection
retry_of	int	FK	nullable; the instruction this one re-issues after the first as rejected on the partner's written confirmation; Finance authorises the one retry (Lifecycle 2d, rule 3)

SETTLEMENT_RETURN

Field	Type	Key	Notes
id	int	PK	
settlement_line_id	int	FK	
returned_amount	decimal		
bank_fee	decimal		
reason	string		
returned_at	datetime		

CONFIRMATION

Field	Type	Key	Notes
id	int	PK	
settlement_line_id	int	FK	
amount_received	decimal		
slip_ref	string		
result	string		full / short
confirmed_by	int	FK	
confirmed_at	datetime		

SHORTFALL

Field	Type	Key	Notes
id	int	PK	

Field	Type	Key	Notes
receiver_id	int	FK	
confirmation_id	int	FK	
amount	decimal		
state	string		open / topped_up
topped_up_settlement_id	int	FK	

BANK_FEE_EVENT

Field	Type	Key	Notes
id	int	PK	
settlement_id	int	FK	
expected_amount	decimal		
actual_amount	decimal		
treatment	string		waived / carry_forward / absorbed
state	string		open / applied
applied_settlement_id	int	FK	

FEE_DECISION

Field	Type	Key	Notes
id	int	PK	
subject_type	string		Deal / Settlement / BalanceConversion
subject_id	int		
fee_kind	string		markup / network / bank
decision	string		apply / waive / override
pct	decimal		
recommended_by	int	FK	Finance — reviews and recommends before Management approves a covered bank fee or a markup above the cap
approved_by	int	FK	Management for a covered bank fee or a markup above max_markup_pct; Finance for the other one-off decisions
reason	string		

MARKUP_SHARE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	
markup_share_rule_id	int	FK	
deal_id	int	FK	
amount	decimal		
state	string		accrued / reconciled / settled / paid)
paid_at	datetime		
direction	string		payable (NPL pays the party) / receivable (the party holds the funds and owes NPL its remainder)
period	string		YYYY-MM; receivables reconcile monthly like rebates
received_amount	decimal		on receivables

PARTNER_REBATE_ACCRUAL

Field	Type	Key	Notes
id	int	PK	

Field	Type	Key	Notes
partner_rebate_rule_id	int	FK	
deal_id	int	FK	
expected_amount	decimal		
received_amount	decimal		
period	string		YYYY-MM
state	string		expected / reconciled / received
statement_ref	string		the partner statement line it reconciled against

LOSS_EVENT

Field	Type	Key	Notes
id	int	PK	
side	string		collection / settlement
collection_id	int	FK	when side = collection
settlement_line_id	int	FK	when side = settlement
amount	decimal		≤ the verified or released amount
reason	string		account_frozen / cash_short / wrong_chain / other
occurred_at	datetime		
sender_share	decimal		
npl_share	decimal		
partner_share	decimal		
client_share	decimal		shares sum to amount
state	string		open / approved / posted
approved_by	int	FK	Management
pool_funded	decimal		the part of the make-good taken from NPL's pool at the partner
wallet_funded	decimal		the part transferred from NPL's own wallet to the partner; what neither covers is the unfunded part, posted as MAKEGOOD / LOSS_PAYABLE until NPL's funds land
funding_reference	string		the partner's credit note or the own-wallet transfer behind the make-good

LOSS_RECOVERY

Field	Type	Key	Notes
id	int	PK	
loss_event_id	int	FK	
party	string		sender / partner / client
amount	decimal		≤ the matching share on the LOSS_EVENT
state	string		agreed / received — a client share is received at once (it is a reversal of the make-good, never a payment). Each state change posts one LEDGER_TRANSACTION naming this row: agreed raises LOSS_RECEIVABLE against EXP_LOSS (or, for the client, reverses part of the make-good), received clears the receivable into POOL or WALLET
received_at	datetime		
reference	string		the partner's credit, the sender's transfer, the client's agreement
approved_by	int	FK	Management (approve_loss_split)

B.5 Ledger & controls

LEDGER_TRANSACTION

Field	Type	Key	Notes
id	int	PK	
source_type	string		Collection / Conversion / BalanceConversion / Settlement / SettlementReturn / Confirmation / BankFeeEvent / Reroute / MarkupShare / Rebate / LossEvent / LossRecovery / Invoice / EarningsOffset / Adjustment / Opening (Opening: one transaction per OPENING_ENTRY row, on the opening date only, Pattern R; Reroute with phase post_conversion posts Pattern O)
source_id	int		
event	string		the step of the source the transaction records (released, confirmed, returned, booked, reversed ...); (source_type, source_id, event) is unique, so one source event posts once (Ledger Posting Design rule 1b)
description	string		
posted_at	datetime		
posted_by	int	FK	
reversal_of_id	int	FK	
corrects_period	string		nullable; YYYY-MM, the month a correction belongs to when it is posted in a later month (Ledger Posting Design, rule 11a); reports by deal date use it, reports by posting date ignore it
idempotency_key	string		unique; the key of the action that created the transaction, so a retried request on a new transport call is refused and returns the first result (Ledger Posting Design rule 1b)

LEDGER_POSTING

Field	Type	Key	Notes
id	int	PK	
ledger_transaction_id	int	FK	
ledger_account_id	int	FK	
side	string		debit / credit
amount	decimal		
currency	string		
ownership_tag	string		client / company
cost_component	string		principal / earnings / rebate / partner_cost / bank_fee / network_fee / variance / share / rounding / fx_timing / cutoff_timing / loss / resale
receiver_group_id	int	FK	nullable; set on every CL.PROJECT.<CCY>.PAYABLE line: the leg's group at collection and conversion, the paying leg's group on a release, return or short confirmation (SETTLEMENT_LINE.deal_id), the served group on a balance conversion, the deal's group on a loss share or an earnings offset — so GROUP_ENTITLEMENT is a sum over postings

LEDGER_ACCOUNT

Field	Type	Key	Notes
id	int	PK	
code	string		OWNER.HOLDER.CCY.PURPOSE
ownership	string		client / company
holder_type	string		PartnerConfig (incl. the own desk) / OwnWallet / Receiver (also the vendor of VENDOR_PAYABLE) / Sender (also LOSS_RECEIVABLE, RESALE_RECEIVABLE) / MarkupShareParty / Project / None
holder_id	int		

Field	Type	Key	Notes
currency	string		
purpose	string		COLLECTED / HELD / DUE / PAYABLE / SHORTFALL / SUSPENSE / CREDIT / MAKEGOOD / POOL / WALLET / EARN_GROSS / EARN_REBATE / RESELL_REVENUE / RESELL_COST / EXP_PARTNER / EXP_BANKFEE / EXP_NETWORK / EXP_ROUNDING / EXP_LOSS / VAR_CONVERSION / VAR_CUTOFF / VAR_RATE_HONOUR / VAR_FX_TIMING / SHARE_PAYABLE / SHARE_RECEIVABLE / REBATE_RECEIVABLE / LOSS_RECEIVABLE / LOSS_PAYABLE / VENDOR_PAYABLE / RESALE_RECEIVABLE / REMIT_CLAIM / FX_RESULT / OPENING_BALANCE (OPENING_BALANCE, equity: the other side of every opening entry that is NPL's own or owed to or by NPL on the opening date, posted once, Pattern R)
kind	string		asset / liability / income / expense / equity (OPENING_BALANCE); FX_RESULT is income or expense by sign, per currency

USER

Field	Type	Key	Notes
id	int	PK	
email	string		Google Workspace
role_id	int	FK	ROLE free-text role)
status	string		
person_ref	string		the natural person behind the account, the same value on every account that person holds (a Finance user's Finance account and the Operations-role account they use for cover); APPROVAL.approved_by is refused when its person_ref equals requested_by's; shown on the users-and-access report

APPROVAL

Field	Type	Key	Notes
id	int	PK	
subject_type	string		
subject_id	int		
action	string		
requested_by	int	FK	
approved_by	int	FK	never equal to requested_by
escalated_to	int	FK	
state	string		
reason	string		
record_version	int		the version of the subject record the approver saw; the approval is applied only if it equals the record's current version, so two approvers at once leave one winner and a changed record is re-approved (Lifecycle & Approvals 6a)

AUDIT_LOG

Field	Type	Key	Notes
id	int	PK	
record_type	string		the record the row is about, resolved through TYPE_REGISTRY; Extract is the production extract's own record (counts per table, the test accounts left out, the check results)
record_id	int		
field	string		
old_value	string		
new_value	string		
actor_id	int	FK	

Field	Type	Key	Notes
at	datetime		

RECORD_LOCK

Field	Type	Key	Notes
id	int	PK	
record_type	string		
record_id	int		
locked_by	int	FK	
locked_at	datetime		
lock_expires_at	datetime		locked_at plus the project's lock_idle_minutes (THRESHOLD, default 15), moved forward on every save; a lock past this time is treated as released
released_at	datetime		nullable; set on save, cancel, expiry or clearance; the row is kept, never deleted
cleared_by	int	FK	nullable; USER, Management, when the lock rather than released by its holder
release_reason	string		saved / cancelled / expired / cleared; every value is also written to AUDIT_LOG

ROLE

Field	Type	Key	Notes
id	int	PK	
code	string		operations / finance / management
name	string		
sees_economics	bool		false for operations: no derived-economics fields on any screen, export or message
status	string		

ROLE_RIGHT

Field	Type	Key	Notes
id	int	PK	
role_id	int	FK	
action	string		a controlled action: price_quote, extend_quote_validity, cross_group_payout, cover_bank_fee, enable_own_desk, approve_loss_split, screening_second_look, earnings_offset, bank_details_approve, send_email_otp, enter_email_otp_code, record_customer_rejection, set_email_otp_settings, …
may_initiate	bool		
may_approve	bool		
escalates_to_role_id	int	FK	nullable; the role added on escalation

EMERGENCY_ACTION

An immediate control applied by Finance and ratified by Management after the fact: a partner suspended, a destination frozen, or a partner's cutoff overridden for a day. It takes effect the moment it is applied, on every deal and settlement it touches, and is lifted if Management does not ratify it within one business day. It is the record behind the controlled action emergency_suspend (Lifecycle & Approvals, section 5). While state = applied or ratified, a partner_suspend blocks new pricing, conversion instructions and settlement releases on every partner pair of the partner configuration; a destination_freeze blocks settlement lines to the account; a cutoff_override replaces the effective cutoff for that day. Every state change is written to AUDIT_LOG; deals in flight follow the three mid-deal cases of the Lifecycle & Approvals document.

Field	Type	Key	Notes
id	int	PK	
kind	string		partner_suspend / destination_freeze / cutoff_override
subject_type	string		PartnerConfig (partner_suspend) / DestinationApproval (destination_freeze) / Partner (cutoff_override)

Field	Type	Key	Notes
subject_id	int		the row of subject_type; checked by the same generic trigger as the other type-plus-id references (Appendix C)
cutoff_time	time		nullable; the cutoff that applies instead, for kind cutoff_override; mirrors a PARTNER_CUTOFF_OVERRIDE of kind extension or early_close recorded at the same time
reason	string		mandatory
applied_by	int	FK	USER, Finance
applied_at	datetime		takes effect at once
ratified_by	int	FK	nullable; USER, Management, never equal to applied_by
ratified_at	datetime		nullable; must be within one business day of applied_at
lifted_at	datetime		nullable; set when Management lifts it, declines to ratify it, or the ratification window passes
state	string		applied / ratified / lifted

PARTNER_STATEMENT_RECONCILIATION

One row per reconciliation of a partner's statement with the ledger, for one partner configuration, currency and statement date (Ledger Posting Design, Section 20, “The reconciliation record”; Pattern Q). Filled in by Finance at every evening reconciliation of the parallel run and at every month end after go-live; the screen is built in Phase 2 (M7). For every active partner configuration and currency there is a row for every business day of the parallel run and for every month end afterwards; a row with state difference_open stays on the exception list until it is resolved.

Field	Type	Key	Notes
id	int	PK	
partner_config_id	int	FK	the partner as used in the project
currency	string		
statement_date	date		the date the partner's balance is stated for
statement_ref	string		the partner's statement or screenshot, in the file store
partner_balance	decimal		what the partner reports it holds for NPL in this currency
ledger_client_balance	decimal		Σ CL.<PARTNER>.<CCY>.COLLECTED + DUE at the statement moment
ledger_pool_balance	decimal		CO.<PARTNER>.<CCY>.POOL at the statement moment
sent_unconfirmed	decimal		Σ released lines not yet confirmed at that moment (the partner may or may not have deducted them)
difference	decimal		partner_balance – (ledger_client_balance + ledger_pool_balance), stated after allowing for sent_unconfirmed
cause	string		nullable; none / fee_not_booked / timing / in_transit / partner_error / ledger_error
true_up_transaction_id	int	FK	nullable; the LEDGER_TRANSACTION that cleared a difference (a true-up, never an edit)
state	string		agreed / difference_open / resolved
reconciled_by	int	FK	USER, Finance
reconciled_at	datetime		
resolved_by	int	FK	nullable; USER, Finance (Management when a true-up is posted)
resolved_at	datetime		nullable

OPENING_ENTRY

One row per opening balance entered on the opening date (Cutover Approach, Section 4; Ledger Posting Design, Pattern R): what it is, where it was verified, who approved it and the posting it produced. The opening date is the first day of the parallel run; every balance the sheets show is entered as a verified opening entry, posted once against CO.NONE.<CCY>.OPENING_BALANCE on the company side or between the client's jar and the project's promise on the client side, and never again.

Field	Type	Key	Notes
id	int	PK	
project_id	int	FK	
partner_config_id	int	FK	nullable; the partner the balance rests at, when it does
currency	string		
kind	string		client_collected / client_due / sender_credit / shortfall / pool / rebate_receivable / share_receivable / share_payable / loss_receivable / earnings_receivable / vendor_payable / resale_receivable / month_volume / suspense (the part of a partner's balance the client sheets do not allocate, opened on the client side per partner and currency, Cutover Approach 4 step 3a) / held (client money resting in one of NPL's own wallets on the opening date)
receiver_group_id	int	FK	nullable; required for client_due (the entitlement it opens)
sender_id	int	FK	nullable; the sender a client_due or sender_credit line belongs to, so the split by sender is kept
party_id	int	FK	nullable; markup-share party, for share lines
amount	decimal		in currency; for month_volume the month-to-date Σ amount_in in the tier's volume currency
as_of	date		the opening date
sheet_ref	string		the sheet and row it reproduces Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
evidence_ref	string		the partner statement, wallet balance or invoice it against Write-once: the key carries the file's hash, a corrected document is a new row naming the old one (ERD Appendix C).
verified_by	int	FK	USER, Finance
approved_by	int	FK	USER, Management, never equal to verified_by
ledger_transaction_id	int	FK	nullable; the Opening transaction posted for it (none for month_volume, which seeds PROJECT_MONTH_VOLUME)
state	string		listed / verified / approved / posted

TYPE_REGISTRY

The registry the generic reference trigger reads: one row per type value a type-plus-id reference may carry (LEDGER_TRANSACTION.source_type, APPROVAL.subject_type, AUDIT_LOG.record_type, WALLET_SCREENING.subject_kind, DEAL.hold_ref and the others), naming the table the id points into and the column of that table that carries the project. An unknown type value is refused; an id must exist in the table named; a ledger source must belong to the transaction's project. A new type value is a row here, approved by Management, never a code change to the trigger.

Field	Type	Key	Notes
id	int	PK	
reference	string		the referencing table and column, e.g. LEDGER_TRANSACTION.source_type
type_value	string		the value, e.g. Settlement
table_name	string		the table the id points into
project_column	string		the column of that table that carries the project; empty for global tables
critical	bool		true for the ledger sources: the project check applies
status	string		
approved_by	int	FK	Management

Appendix C · Constraints and invariants, and where each is enforced

Every rule stated in prose in this document, in the Configuration Schema (section 6) and in the Ledger Posting Design (section 2) is listed here with the mechanism that enforces it in the deployable schema. Four kinds: a unique key or foreign key, a CHECK on one row, a database trigger (for rules that span rows or tables), or an application validation in the API. The last column says which rules must be at database level, because a screen, a job or a mistake must not be able to get around them; the others are enforced by the API and verified by tests. Implemented in Phase 1 (M2) with the schema; the acceptance test for each is a write that breaks the rule and is refused.

Rule	Mechanism	Database level	Notes
Ledger and audit tables accept inserts only	Privileges (no UPDATE or DELETE for the application role) and a BEFORE UPDATE OR DELETE trigger	must	Architecture note §3; LEDGER_ACCOUNT may only change status to closed
Postings balance per currency and per ownership tag within a transaction	Deferred constraint trigger at commit	must	Ledger Posting Design rule 2
Every CL.PROJECT.<CCY>.PAYABLE line carries receiver_group_id	Trigger on LEDGER_POSTING	must	GROUP_ENTITLEMENT is a sum over these
A transaction names a source record of the stated source_type, and the record exists	Generic type-plus-id trigger (below)	must	rule 1
One transaction per event; a retried event posts nothing	Unique key on the idempotency key of every money-moving action	must	rule 1a; Architecture note §5
A CONVERSION has exactly one owner	CHECK: exactly one of deal_id, balance_conversion_id is set	must	
Approver is never the initiator, on the person	Trigger on APPROVAL comparing USER.person_ref of approved_by and requested_by	must	Roles & Visibility rule 5 and 5a
Approval valid only for the record version seen	Trigger comparing APPROVAL.record_version with the record's current version	must	Lifecycle & Approvals 6a
Versioned configuration is write-once	Trigger refusing UPDATE on FEE_STRUCTURE, FEE_TIER, THRESHOLD, SYSTEM_SETTING, MESSAGE_TEMPLATE, PARTNER_RATE_VERSION, FEE_OVERRIDE except the status column	must	a new value is a new row with a later effective_from
Pricing fields are write-once after priced_at	Trigger on DEAL and DEAL_GROUP	must	Lifecycle & Approvals §2
Exactly one default receiver group per project	Partial unique index on RECEIVER_GROUP(project_id) where is_default	yes	Configuration Schema rule 2

Rule	Mechanism	Database level	Notes
Accounts unique per entity, currency and rail; endpoints unique per entity, currency, network or rail	Unique keys	yes	rule 4
A single-use account or endpoint serves one deal	Unique key on reserved_for_deal_id where usage = single_use	yes	
Bank details are write-once; a change is a new version	Trigger refusing UPDATE on BANK_ACCOUNT_DETAILS except the status column; supersedes_id names the replaced row	must	the version current at release stays what a settlement line resolves to
A bank account, bank endpoint or settlement rail is active only with active bank details of the same rail	Trigger on status change checking the BANK_ACCOUNT_DETAILS row and its rail	yes	Configuration Schema rule 14
rail_fields match the rail's template: required fields present, format rules pass (IBAN check digits, BIC length, sort-code pattern)	Application validation on save against BANK_DETAILS_TEMPLATE	no	Configuration Schema rule 14
Every full read of a bank-details row is logged	Application: the read endpoint writes an AUDIT_LOG row (record_type BANK_ACCOUNT_DETAILS, field read, actor, time)	no	Roles & Visibility rule 10
One open lock per record	Partial unique index on RECORD_LOCK(record_type, record_id) where released_at is null	yes	Lifecycle & Approvals 6a
One reconciliation per partner configuration, currency and statement date	Unique key on PARTNER_STATEMENT_RECONCILIATION	yes	
sender_pays_pct + receiver_pays_pct = pct; min_markup ≤ max_markup; a fixed customer-facing fee names its pct	CHECK on FEE_STRUCTURE and FEE_OVERRIDE	yes	rule 6
partner_fee_pct present exactly for fee_on_market	CHECK on PARTNER_PAIR	yes	rule 5

Rule	Mechanism	Database level	Notes
A bank endpoint is active only with a rail; a wallet's rail equals its network	CHECK on COLLECTION_RECEIVING_ENDPOINT and RECEIVING_ENTITY_ACCOUNT	yes	rule 7
A cutoff override is one-day or weekday, never both; an extension names the partner's agreement	CHECK on PARTNER_CUTOFF_OVERRIDE	yes	rule 10
quote_freshness_min < quote_validity_min	CHECK on PROJECT	yes	rule 11
A project prices deals only when active	Trigger on DEAL insert checking PROJECT.state	yes	Configuration Schema 1a
Loss shares sum to the loss amount; a recovery never exceeds its share	Trigger on LOSS_EVENT and LOSS_RECOVERY	yes	
Σ DEAL_FEE_TIER portions = amount_in	Trigger	yes	
needs_bank_lock requires the partner's cutoff and time zone; a partner pair's currency pair belongs to the project; a wallet endpoint's network is one the partner offers	Application validation on save, re-checked by the project validation report	no	cross-table rules of Configuration Schema §6; the validation report lists them
Wallet addresses are checksum-valid per network	Application validation	no	
Σ markup shares of one base ≤ 100 %; tiers have one row at volume 0 and unique thresholds	Application validation; unique key on FEE_TIER(fee_structure_id, min_monthly_volume)	partly	
State transitions follow the state machines	Application (every state change is a named action)	no	Architecture note §5; tested per transition
Every reference to another table is a foreign key	FOREIGN KEY on every column the dictionary marks FK	yes	Appendix B
No row of the production image belongs to a test project, directly or through a type-plus-id reference	The post-extract scan over the dependency closure (below), run before the one commit of the load and after every restore	must	Cutover Approach §7; Test Strategy G-FILTER, G-RESTORE

Rule	Mechanism	Database level	Notes
Zero orphan references after an extract or a restore: every FOREIGN KEY and every type-plus-id reference resolves	FOREIGN KEYs checked at the commit of the load (constraints deferred); the registry trigger's check re-run as a scan over the seven references	must	G-FILTER, G-RESTORE
A type value is one the registry knows, the id exists in the table the registry maps it to, and a ledger source belongs to the transaction's project	TYPE_REGISTRY table read by the generic trigger; CHECK against the registry; for LEDGER_TRANSACTION the trigger also reads the source row's project through the registry's project column	must	option b, strengthened (below)
Evidence references are write-once and content-addressed	Trigger refusing UPDATE of every evidence_ref and statement_ref column once set; the object key carries the file's SHA-256; bucket versioning with object lock (Architecture §3)	must	below; D-POLY11
A leg on hold accepts no money-moving write	Trigger on CONVERSION, BALANCE_CONVERSION, SETTLEMENT_LINE, the reroute forward and the refund disposition checking DEAL.hold_kind	must	Lifecycle 2c; DEAL.hold_kind, hold_ref
One open external instruction per promise	Partial unique index on the promise (settlement line, balance) where ack_status is instructed, acknowledged or unknown	must	Lifecycle 2d; ack_status and instruction_ref
A screening decision is write-once; a reject is lifted only by a new screening row	Trigger refusing UPDATE of WALLET_SCREENING.decision	must	Roles & Visibility rule 11; Configuration Schema §4
A flagged or breached rate version prices nothing	Trigger on DEAL and CONVERSION insert checking PARTNER_RATE_VERSION.status = usable	must	Calculation Specification 2a; Lifecycle Figure 3; PARTNER_RATE_VERSION.status flagged
A dormant partner pair prices nothing and accepts no collection or settlement line	Trigger on DEAL, COLLECTION and SETTLEMENT_LINE insert checking PARTNER_PAIR.status	yes	Scenario Catalogue Part 1; PARTNER_PAIR.status dormant
A sender's source is a bank-details row or a wallet network and address, exactly one of the two	CHECK on COLLECTION_SENDING_ENTITY: (bank_account_details_id IS NOT NULL) <> (wallet_network IS NOT NULL AND wallet_address IS NOT NULL), and wallet_network and wallet_address set together	must	Configuration Schema §4 COLLECTION_SENDING_ENTITY (NPL, 28 October 2026)
Email OTP: one open code per quote issue	Partial unique index on CUSTOMER_APPROVAL(quote_package_id) where status = issued	must	Lifecycle 2e

Rule	Mechanism	Database level	Notes
Email OTP: a code validates once; the decision is write-once	Trigger refusing UPDATE of status once approved / rejected / expired / voided / locked, and of decided_at, decided_by, evidence_ref, reject_reason	must	Lifecycle 2e; Roles rule 13
Email OTP: no collection without the customer's approval of the current issue	Trigger on COLLECTION insert: when PROJECT.email_otp_enabled, an approved CUSTOMER_APPROVAL exists for the group's current (not superseded) quote issue	must	Lifecycle 2e; Test Strategy S-OTP
Email OTP: a new issue or a change of approver address voids the open code	Trigger on QUOTE_PACKAGE insert (superseded_by set) and on SENDER.email_otp_approver_email change: open rows of the old issue, and that sender's open rows on the old address, set to voided	must	Lifecycle 2e
Email OTP settings are complete and approved before use, and never edited directly	send_email_otp is refused for a sender whose SENDER.email_otp_approver_email or chat columns are empty or unapproved while PROJECT.email_otp_enabled (the quote cannot be sent; the screen says why; no fallback); the columns change only through the controlled action set_email_otp_settings with its APPROVAL (Finance); the API refuses send_email_otp for that sender while such an APPROVAL is pending	must	Configuration Schema rule 16; Roles rule 13

Type-plus-id references: option b, one generic trigger

NPL chose option b on 19 October: the type-plus-id references stay as they are and one generic database trigger checks, on insert and update, that the id exists in the table the type column names, and refuses the row otherwise. The trigger is applied to all seven such references: `LEDGER_TRANSACTION` (source_type, source_id), `LEDGER_ACCOUNT` (holder_type, holder_id), `APPROVAL` (subject_type, subject_id), `AUDIT_LOG` (record_type, record_id), `RECORD_LOCK` (record_type, record_id), `FEE_DECISION` (subject_type, subject_id) and `EMERGENCY_ACTION` (subject_type, subject_id); `DEAL.funding_ref_id` is checked against the table funding_source names in the same way. The trade-off accepted: the database cannot cascade or report these as foreign keys, so a referenced row can never be deleted (none of these tables deletes), and a type value is added by a row in the TYPE_REGISTRY table the trigger reads (type value → table name → the column that carries the project, where one exists), never by editing the trigger; a value the registry does not know is refused. The acceptance test inserts a row whose id does not exist for each of the seven references and expects a refusal. The four discriminator-plus-column references (`WALLET_SCREENING.subject_kind`, `LOSS_EVENT.side`, `RECEIVER.kind`, `CONVERSION`'s one owner) are ordinary foreign keys with a CHECK that exactly the right column is set.

Critical sources: three checks, not one

For the ledger (LEDGER_TRANSACTION.source_type, source_id) existence is not enough, because an id that happens to exist in another table would pass an existence check under the wrong type. Three checks apply to every ledger source, and the first two to the other six references: (1) the type value is in TYPE_REGISTRY; (2) the id exists in the table the registry maps the type to; (3) for a ledger source, the source row's project (read through the registry's project column: the deal's project for a Collection, Conversion or Settlement line, the balance conversion's, the reroute's, the opening entry's) equals the transaction's project, so a Settlement id of another project is refused as firmly as a missing one. The negative tests D-POLY8 to D-POLY10 insert, for each reference, an unknown type, an id that exists only in another table and a ledger source of another project, and

expect a refusal from the database and from the API; a restore into a scratch instance re-runs the same three checks as a scan and reports every broken link (G-RESTORE).

Immutable evidence references

Six columns name a file: COLLECTION.evidence_ref, SETTLEMENT_LINE.evidence_ref, WALLET_SCREENING.evidence_ref, OPENING_ENTRY.evidence_ref, BANK_ACCOUNT_DETAILS.evidence_ref and PARTNER_STATEMENT_RECONCILIATION.statement_ref. Each is write-once (a trigger refuses an update once the value is set; a corrected document is a new row that names the old one, as everywhere in the model) and content-addressed: the object key carries the file's SHA-256, the bucket keeps every version with an object lock, and the application rejects an upload whose hash does not match the key it is stored under. A reference therefore resolves to exactly the bytes that were attached, or to nothing; it can never resolve to a changed file. The restore audit re-hashes every object the extracted rows name and reports a missing or altered one (D-POLY11, G-RESTORE).

The extract closure and the clean-production image

The Cutover Approach (Section 7) builds production from staging by extracting the real projects and everything that belongs to them, deleting nothing on staging. The reviewer asked for the closure to be published and for the integrity of the image to be checked by the database, not assumed. The closure is the dependency graph of Appendix B read from PROJECT outward, in five classes:

Class	Tables	Rule in the extract
Roots	PROJECT where is_test is false; the CLIENT rows those projects belong to	Copied. A client with only test projects is not.
Project-owned	The 17 tables with project_id and every table reached from them by foreign key: parties and allow-lists, receiving entities and accounts, the partner configuration block (pairs, entities, endpoints, sending entities, rails, destination approvals, fee structures, tiers, overrides, rebate and share rules), deal groups and legs, quote packages and extensions, collections, conversions, reroutes, balance conversions, credits, settlements and lines, returns, confirmations, shortfalls, bank-fee events, accruals, loss events and recoveries, earnings receivable and offsets, opening entries, screenings, bank account details, reconciliations	Copied when the path by foreign keys leads to a real project; never when it leads to a test project. A row with no path to any project is in the next class.
Shared, copied whole	PARTNER, HOLIDAY, PARTNER_CUTOFF_OVERRIDE, RATE_SOURCE, MARKET_RATE, RATE_COMPARISON, PARTNER_RATE_VERSION, SYSTEM_SETTING, the global MESSAGE_TEMPLATE rows, BANK_DETAILS_TEMPLATE, MARKUP_SHARE_PARTY, the platform CURRENCY_PAIRs, ROLE, ROLE_RIGHT	Copied whole. USER is copied without the test accounts, which are a named list recorded in the extract's audit row.
Type-plus-id tables	LEDGER_TRANSACTION with its LEDGER_POSTING rows, LEDGER_ACCOUNT, APPROVAL, AUDIT_LOG, RECORD_LOCK, FEE_DECISION, EMERGENCY_ACTION	A row is copied if and only if the record its type and id name is in the extract, resolved through TYPE_REGISTRY; an AUDIT_LOG row on a shared row is copied, one on a test record is not. The reviewer's example (an audit row for a test deal) is therefore excluded by construction, not by a filter on PROJECT.

Class	Tables	Rule in the extract
Files	The objects named by the extracted rows' evidence_ref and statement_ref columns	Copied by key, re-hashed on arrival; nothing else from the staging bucket.

Ids are never renumbered. Every table is copied with its ids, order and content unchanged, so a reference in production equals the same reference on staging and the two images can be compared row by row.

The checks on the image, run by the extract before its one commit and again after every restore from backup (Cutover 8a), are the automated scan the reviewer asks for: (1) every FOREIGN KEY resolves — the load runs with constraints deferred and commits once, so a dangling key fails the whole load; (2) every type-plus-id reference resolves through the registry, with the three checks above; (3) the closure inverted from the test projects reaches zero rows of the image; (4) the count per table equals staging's count of real-project rows, computed by the same closure on staging; (5) the ledger identities of the Ledger Posting Design (Section 20) hold and every balance per project, partner and currency equals staging's; (6) every evidence object is present and its SHA-256 equals its key; (7) the extract writes one AUDIT_LOG row in production (record_type Extract) naming the staging snapshot, the counts per table, the test accounts left out and the result of each check. Any check that fails leaves production empty (Cutover, rollback).

Audit retention. The production AUDIT_LOG and ledger accept inserts only and are never pruned. The audit rows of the test projects are not in production; they are in the staging snapshot, which is kept read-only as the record of the parallel run, test projects included, for the retention period NPL sets (a SYSTEM_SETTING; proposed: the same period as the ledger's). Nothing of the run is therefore lost: production holds the real projects' audit from their first staging entry, and the snapshot holds the rest.

Appendix D · Locking and indexes

At ten deals a weekday the model is a few thousand rows a month, so locking is about correctness and indexes are about the few queries that scan: balances, reconciliation and search. Implemented in Phase 1 (M2); the concurrency tests of the go-live acceptance exercise the locks.

Lock	Taken when	Effect
Row lock on the record of a controlled action	For the duration of the action's database transaction (SELECT … FOR UPDATE on the DEAL, SETTLEMENT, CONVERSION or configuration row)	Two users acting on the same record are serialised; the second sees the first's result and, if the state changed, is refused by the version check (6a)
Lock on the APPROVAL row while it is approved or rejected	During the approver's action	Two approvers on one request: the first commit wins, the second is refused
Lock on the project's month-volume row	While a quote is priced	Two quotes priced at once read the counter in order, so a tier split is computed on the true month-to-date figure (Calculation Specification 5a)
Job lock row per job	From the start to the end of a job run	Two runs of a job never overlap (6a); a run that finds the lock held exits and is recorded as skipped
Edit lock (RECORD_LOCK)	While a person has a record open for editing	Advisory, with expiry and clearance (6a); it prevents two people editing one record, not two transactions

Index	Serves
`LEDGER_POSTING (ledger_account_id, ledger_transaction_id)`	every balance view, the ledger explorer

Index	Serves
`LEDGER_TRANSACTION (source_type, source_id); (posted_at)`	tracing a posting to its event; reports by posting date
`LEDGER_POSTING (receiver_group_id) where not null`	GROUP_ENTITLEMENT
`DEAL (deal_group_id); DEAL (state, project_id); DEAL (priced_at)`	the per-role lists, the month-volume counter, reports by deal date
`COLLECTION (deal_id); SETTLEMENT_LINE (settlement_id); SETTLEMENT_LINE (outcome, expected_confirmation_by)`	deal screens; the sent-not-confirmed report and the exception list
`APPROVAL (state, subject_type, subject_id); APPROVAL (requested_by, state)`	the approvals queue; own requests
`AUDIT_LOG (record_type, record_id, at)`	the audit trail of a record
`PARTNER_RATE_VERSION (partner_pair_id, status, expires_at); MARKET_RATE (currency_pair_id, captured_at)`	pricing, the day's comparison, the two-week average
`PARTNER_STATEMENT_RECONCILIATION (partner_config_id, currency, statement_date) unique`	reconciliation history
`WALLET_SCREENING (network, address, screened_at)`	the decision that binds later deals on the same address
`USER (person_ref)`	the person-level approver check

No other index is planned for P0; one is added when a query is measured to need it, never in advance.